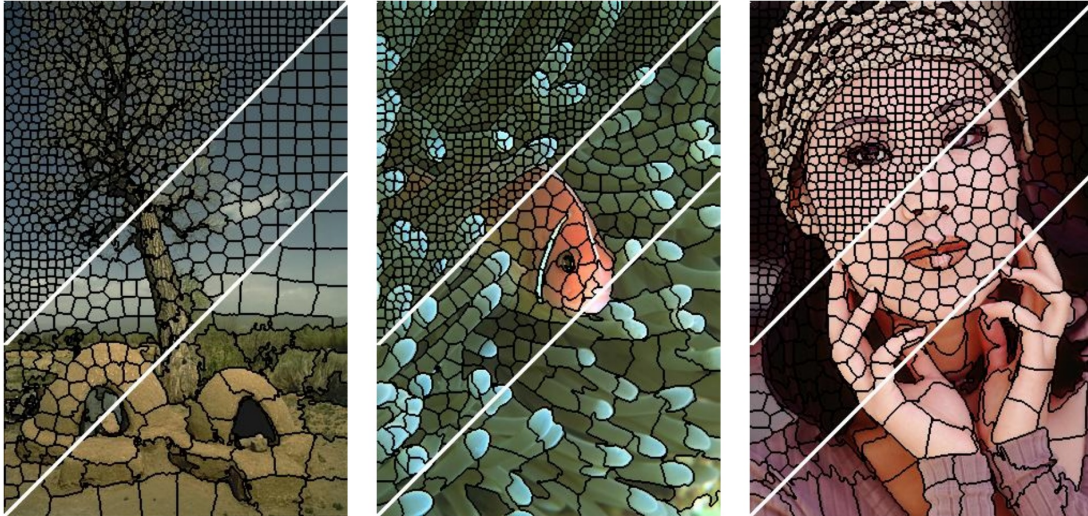


Clustering



Mehrtash Harandi

MEHRTASH.HARANDI@MONASH.EDU

Department of Electrical and Computer Systems Engineering, Monash University

Contents

1	Introduction	4
1.1	Clustering and the Cholera Outbreak in London	6
2	K-means	7
2.1	K-means Problem	9
2.2	Algorithm	10
2.3	Initialization	12
2.4	Determining the number of clusters	13
2.5	Convergence [†]	13
2.6	K-Medoids Algorithm [†]	14
3	Gaussian Mixture Models (GMMs)	15
3.1	From K-Means to GMMs	17
4	Beyond K-means and GMMs	24
5	Some Applications	26
5.1	Image Segmentation and Compression	26
5.2	Vector Quantization	29
5.3	Superpixels	30
6	Exercises	33
6.1	K-means	33
6.2	GMM	35
7	You Should	38

Disclaimer. I have prepared these notes to supplement the course “Computer Vision” delivered at Monash University. I cannot make any guarantees that these notes are free of typos or other more serious errors. I use language models such as [ChatGPT](#) in preparing my notes (why not?). If a language model is used, I have checked the generated text to the best of my knowledge to confirm and validate it. This, however, does not necessarily translate to a formidable guarantee as, knowledge-wise, I cannot arm-wrestle with Google and Wikipedia and alikes! In preparing my notes, I might have occasionally borrowed some text from books and other resources, not because I am lazy but because I have found the explanation better than what I could think of. I have tried my best to acknowledge this, but it is possible that something slipped my scrutiny.

1. Introduction

Data is everywhere. Surveillance cameras capture videos 24/7. Every time you make a phone call, your name and location are recorded. Your browsing pattern is recorded when surfing the web. Your financial transactions are recorded. Telescopes, satellites, and observatories generate massive amounts of data, to name a few. That said, data by itself is useless. The hidden gem in data is “information”. The objective of machine learning is to extract relevant information from data and make it available to the user.



Figure 1: Be a good friend!

What is Learning? Machine learning can be broadly defined as *computational methods* that use experience to improve their performance, such as accurate predictions. Here, experience refers to the past information available to the learner, which typically takes the form of electronic data collected and made available for analysis.

This definition captures the essence of what it means for a computer system to “learn” from data. In traditional programming, a developer writes code that instructs the computer how to perform a certain task. In machine learning, however, the computer is not explicitly programmed with the solution to a problem. Instead, it uses experience (*i.e.*, data) to identify patterns and relationships that allow it to make accurate predictions or decisions.

Types of Learning. Learning is an interactive process between the learner and the environment, and different types of learning tasks can be classified based on the nature of this interaction. One of the primary distinctions is between supervised and unsupervised learning.

To illustrate this difference, let’s consider two tasks. The first task involves learning to *classify images of cats and dogs*. In this scenario, the learner is provided with a set of training images labeled as either cats or dogs, and the goal is to develop a rule that can accurately label new images as either cats or dogs. The second task is *anomaly detection* for surveillance, where the learner must identify *unusual* events in a large collection of videos with no labels.

In supervised learning, the training data contains significant supervisory information, such as the cat/dog labels in the previous example. However, this information is often missing for unseen test examples, which are the examples that the learner’s expertise must be applied to.

In unsupervised learning, the learner processes input data without any supervisory information. The goal is to extract a **summary** or **compressed** version of the data. Clustering a data set into subsets of **similar objects** is a typical example of such a task, which will be the topic of this chapter.



Figure 2: An example of anomaly detection from video sequences.

Notations. We will use boldface font to denote vectors and matrices. For example, $\mathbf{x}$ and $\mathbf{A}$ denote a vector and a matrix, respectively. Vectors are column vectors, and their dimensionality is shown, if needed, by specifying their domain. For example, a vector shown by $\mathbf{x} \in \mathbb{R}^n$ tells us that $\mathbf{x}$ has n dimensions, and each dimension is a real number. Note that when coding an algorithm, you may want to store vectors as row vectors to simplify computations. That is fine, but to be rigorous and have a common language, we follow the practice in mathematics and consider column vectors. Similarly, the notation $\mathbf{A} \in \mathbb{R}^{n \times m}$ means that $\mathbf{A}$ is a real matrix of size $n \times m$ (i.e., n rows and m columns). The [transpose](#) of a matrix is denoted by $\top$. The [Euclidean distance](#), which is a generalization of what you have seen in $\mathbb{R}^2$, between two vectors $\mathbf{x}, \mathbf{z} \in \mathbb{R}^n$ is defined as

$$\|\mathbf{x} - \mathbf{z}\| = \sqrt{\sum_{i=1}^n |x[i] - z[i]|^2}. \quad (1)$$

Here, $x[i]$ is the element at location i of vector $\mathbf{x}$.

Example 1 Consider the following three points in $\mathbb{R}^3$:

$$\mathbf{A} = \begin{pmatrix} 2 \\ 5 \\ 7 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} -3 \\ 4 \\ 9 \end{pmatrix}, \quad \mathbf{C} = \begin{pmatrix} 1 \\ -2 \\ 5 \end{pmatrix}.$$

Which two points are the closest to each other?

To answer this question, you need to calculate the distance between each pair of points and find the minimum distance. The two points with the minimum distance are the closest to each other. The distance between two points $P = (x_1, y_1, z_1)$ and $Q = (x_2, y_2, z_2)$ in $\mathbb{R}^3$ is given by (see Equation (1)):

$$d(P, Q) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

Using this formula, we can calculate the distances between each pair of points:

$$d(A, B) = \sqrt{(-3 - 2)^2 + (4 - 5)^2 + (9 - 7)^2} = \sqrt{30}$$

$$d(A, C) = \sqrt{(1 - 2)^2 + (-2 - 5)^2 + (5 - 7)^2} = \sqrt{54}$$

$$d(B, C) = \sqrt{(1 + 3)^2 + (-2 - 4)^2 + (5 - 9)^2} = \sqrt{68}$$

Therefore, the two points that are closest to each other are A and B , since they have the smallest distance of $\sqrt{30}$.

Question. To find the closest point using the Euclidean distance, do we need to consider the square root function? Why?¹

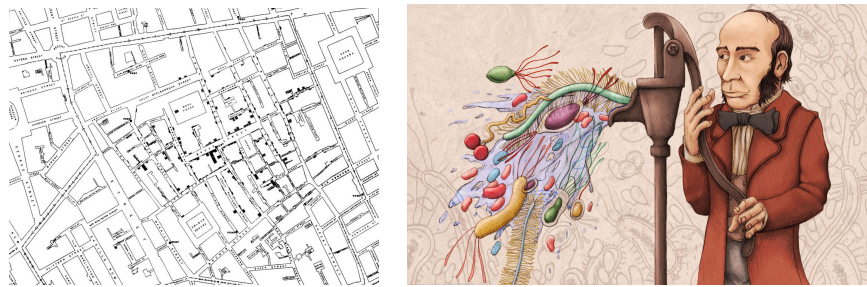


Figure 3: John Snow (1813 – 1858) was an English physician and is considered one of the founders of modern epidemiology. Snow used a dot map to illustrate the cluster of cholera cases around water pumps and identified the connection between the quality of the water source and cholera cases. The illustration on the right from John Snow's work is from the Guardian and is done by Alexander Bertram Powell.

1.1. Clustering and the Cholera Outbreak in London

Clustering is an unsupervised data analysis technique used to unveil the underlying structure of a set of data points by dividing them into (potentially) non-overlapping groups called **clusters**. The goal is to achieve high intra-cluster similarity (similarity among the points within a cluster) while minimizing inter-cluster similarity (similarity among points in different clusters). One of the earliest known applications of clustering dates back to John Snow's study in 1854, where he identified the source of a cholera outbreak in London (see Figure 3). The prevailing scientific explanation at the time, the *miasma theory*, held that diseases such as cholera were caused by "bad air". The germ theory of disease had not yet been developed, so Snow did not understand the mechanism by which the disease was transmitted. By talking to local residents and mapping the cases onto a map of

¹We do not necessarily need to consider the square root function to find the closest point using the Euclidean distance. The reason is that the square root is a monotonic function, meaning it preserves the ordering of distances. In other words, if we have two distances d_1 and d_2 , where $d_1 < d_2$, then $d_1^2 < d_2^2$. Therefore, we can compare distances without taking the square root. This is known as the squared Euclidean distance, obtained by omitting the square root from the Euclidean distance formula. The squared Euclidean distance is often used in machine learning algorithms, where the computational cost of taking square roots can be significant for large datasets. However, it is worth noting that the squared Euclidean distance does not have the same units as the Euclidean distance, so it is not directly comparable to distances in the physical world.

London, Snow discovered that the source of the outbreak as public water pumps. Using statistical analysis, he demonstrated the connection between the quality of the water source and cholera cases. As a result of Snow's work, the local council disabled the pump, which led to the end of the outbreak.

In computer vision, clustering techniques are used to address a broad set of problems, from image segmentation to image coding and generation. In what follows, we will study two clustering algorithms, namely *the K-means* and *the GMM*. Both algorithms are indeed the workhorses of many developments, so you should consider them as meta-algorithms with applications going well beyond our discussions in this chapter.

2. K-means

K-means clustering is a celebrated and elegant approach for partitioning a data set into K distinct, non-overlapping clusters. To perform K-means clustering, we must first specify the desired number of clusters K ; then the K-means algorithm will assign each data point to exactly one of the K clusters. Furthermore, K-means describes each cluster with a cluster center for us.

To understand the K-means algorithm, consider Figure 4(a). Our goal is to group the points into two clusters. If you look at the plot, you may quickly suggest the groups shown in Figure 4(b). Now, we can ask ourselves a simple question, *how did we come up with this solution?* What we would like to achieve by asking this question is to figure out a set of instructions that we can code into an algorithm to cluster the data in a way that is similar to how a human would do the job. However, explaining the process in our own words can be surprisingly difficult!

Homer to the rescue. Clustering is a complex problem, but we can simplify it by cheating a little. To do this, we'll turn to the god of wisdom, Homer², for a clue. Homer tells me that points marked by squares in Figure 4(c) are cluster centers (*i.e.*, cluster representatives). Of course, Homer speaks in his own language, something like, "Doh, the center of each cluster is marked by a square," but it is probably not that important. Using this information, can you tell me which points are in the yellow cluster and which ones are in the pink cluster?



This is simple. All we need to do is to use the Euclidean distance discussed in Equation (1) and measure the distance of each point to the cluster centers. Once we obtain that, we can assign each point to its nearest cluster. Easy, right? This is shown for one data point in Figure 4(c). But hold on; we know that Homer is not always right. So how about we show him our results, depicted in Figure 4(d)? If we do that, Homer will say, *Shouldn't the centers be the center of each cluster?* Hmm, I should give it to Homer; he is right again. We can compute the center quite easily. Once we know to which cluster each point is assigned to, we can compute the cluster centers by **averaging** all the points within the cluster. This will give us the new cluster centers, and we can repeat the process described above several times, hoping that after a few iterations, we will reach a consensus and the change in assignments/centers will become negligible to stop. If what I described above makes sense to you, congratulations, you have developed one of the fundamental algorithms in data science (to a good degree), the K-means algorithm.

²I literally mean Homer, not [Skynet](#) (aka ChatGPT).

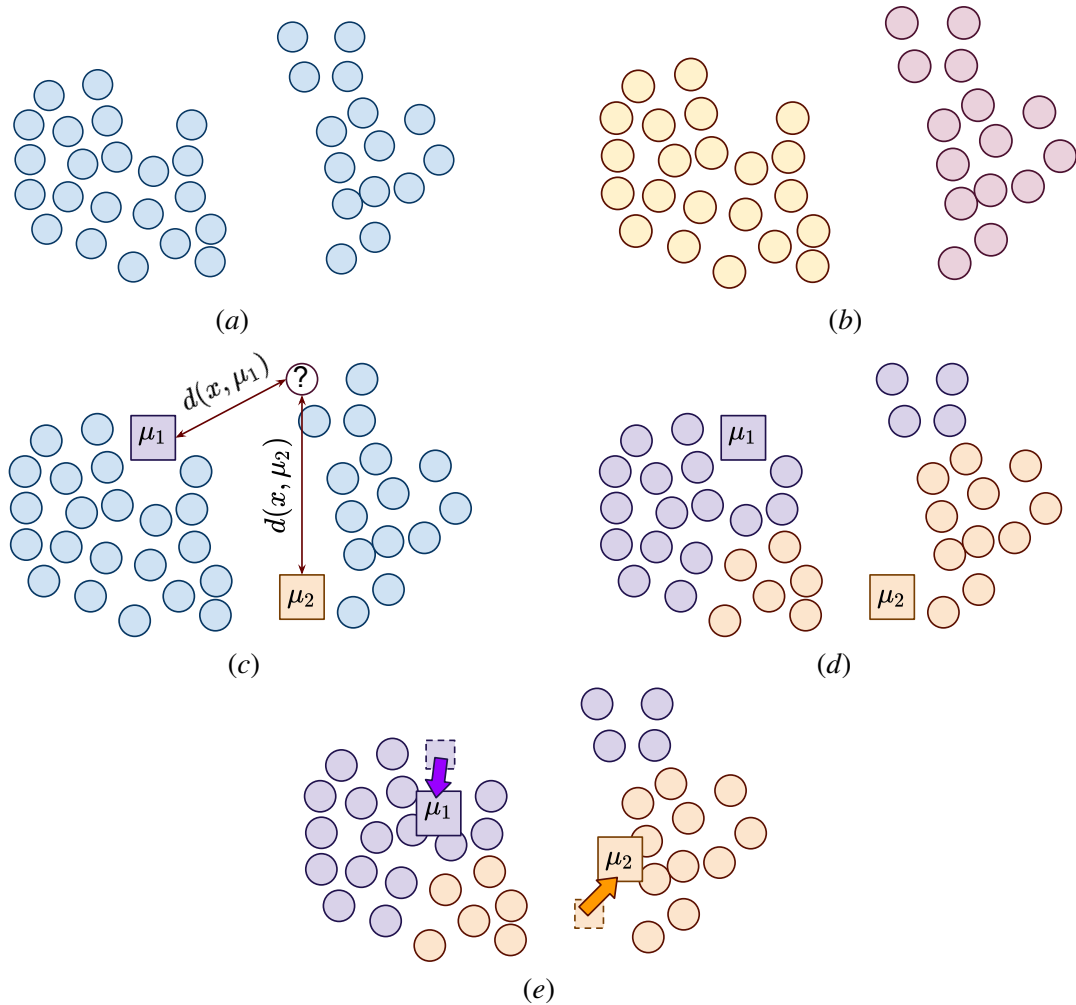


Figure 4: Intuitive illustration of clustering using the K-means algorithm. **(a)** Given a set of samples (blue circles). Our goal is to group them here into $K = 2$ clusters. **(b)** For humans and in low-dimensional spaces, we can readily cluster data points by visualizing their placements if data points are a handful. **(c)** If an oracle tells you that the cluster centers are μ_1 and μ_2 , identifying to which cluster a sample belongs is easy. We just need to compute the distance to the cluster centers and assign the point to the nearest cluster. The result of assigning points to the closest clusters is shown in **(d)**. **(e)** If we know to which cluster data points are assigned to, we can find cluster centers easily by averaging.

2.1. K-means Problem

Given a set of m data points (instances) $\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_m\}$ with $\mathbf{x}_i \in \mathbb{R}^n$ and an integer K , the **K-means problem** is to determine a set of K centroids $\mathcal{C} = \{\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K\}$ with $\boldsymbol{\mu}_i \in \mathbb{R}^n$, so as to minimize the following error function:

$$\mathcal{L}(\mathcal{C}) \triangleq \sum_{i=1}^m \sum_{k=1}^K \nu_{i,k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2, \quad (2)$$

where

$$\nu_{i,k} = \begin{cases} 1 & \text{if } k = \arg \min_j \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2 \\ 0 & \text{otherwise} \end{cases}. \quad (3)$$

Let me explain Equations (2) and (3). Equation (2) is the total error of clustering. It basically tells you that for each sample $\mathbf{x}_i$, you measure its error w.r.t.³ to all cluster centers via

$$e(\mathbf{x}_i) \triangleq \sum_{k=1}^K \nu_{i,k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2.$$

Here, $\nu_{i,k}$ is either 1 or 0, depending on whether cluster center $\boldsymbol{\mu}_k$ is the closest to $\mathbf{x}_i$ among all clusters (see Equation (3)). It is only 1, iff among all clusters, cluster center $\boldsymbol{\mu}_k$ is the closest to $\mathbf{x}_i$ per Equation (3). Therefore, for each data point $\mathbf{x}_i$, we find its closest cluster and compute the error using squared Euclidean distance as $e(\mathbf{x}_i)$. We then add up the errors of all data points to obtain the total error, which is denoted as $\mathcal{L}(\mathcal{C}) = \sum_i e(\mathbf{x}_i)$. This approach allows us to frame the problem as an optimization problem.

We can minimize Equation (2) by selecting the optimal values of $\mathcal{C} = \boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K$. However, there is a catch. This is a combinatorial optimization problem since it is equivalent to finding the partition of the m instances in K groups whose associated set of centers of mass minimizes Equation (2). The number of possible partitions is a [Stirling number of the second kind](#),

$$S(m, K) = \frac{1}{K!} \sum_{i=0}^K (-1)^i \binom{K}{i} (K-i)^m.$$

For example, to group $m = 10$ points into $K = 3$ groups, there are $S(10, 3) = 9330$ possibilities. It is known that finding the globally optimal partition is NP-hard⁴, even for instances in the plane. As such, exhaustive search methods are not useful under this setting, leading to the design of iterative refinement-based algorithms to approximate the solution of the K -means. As the name implies, these algorithms iteratively relocate the data points between clusters until a locally optimal partition is attained. Arguably, the most popular iterative algorithm is the **K -means algorithm**.

³w.r.t. = with respect to

⁴In theoretical computer science, the two most basic classes of problems are P and NP. The class of P includes all problems that can be solved efficiently. An example is adding two numbers. NP includes all problems that given a *solution*, one can efficiently verify whether the solution is correct or not, but it is unknown if the problem has an efficient algorithm for finding the solution. So verifying is easy for NP problems, but finding the solution efficiently is unknown. An example is: given a bunch of numbers, can they be split into two groups such that the sum of one group is the same as the other? Clearly, if one is given a solution (two groups of numbers), it's a no-brainer to verify if the sum is the same.

2.2. Algorithm

The K -means algorithm is an iterative and greedy method that consists of two stages: **Initialization**, in which we set the starting set of K centroids, and an iterative stage, known as **Lloyd's algorithm** (Lloyd, 1982). In the first step of Lloyd's algorithm, each instance is assigned to its closest centroid (**assignment step** or **E-step**), then the set of centroids is updated as the centers of mass of the instances assigned to the same centroid in the previous step (**update step** or **M-step**). Finally, a stopping criterion is verified. The most common criteria are

- If the assignments do not change after an iteration of the algorithm, we stop. That is, we keep a copy of $a_{i,k}$ from the previous iteration and compare it with the current assignments. If the lists match exactly, the algorithm has converged.
- In many instances, one can stop even if the algorithm has not converged completely. In such a case, the algorithm stops if the error depicted in Equation (2) does not change significantly with respect to the previous iteration. To do this, we use the $\mathcal{C}$ and $\mathcal{C}'$, the set of centroids obtained at consecutive Lloyd's iterations, and stop when

$$|\mathcal{L}(\mathcal{C}) - \mathcal{L}(\mathcal{C}')| \leq \varepsilon, \text{ for a fixed threshold } \varepsilon \ll 1. \quad (4)$$

The K -means algorithm can be easily parallelized, which is crucial for meeting its scalability requirements. The time needed for the assignment step is $\mathcal{O}(m \cdot K \cdot n)$, while updating the set of centroids requires $\mathcal{O}(m \cdot n)$ computations and the stopping criterion, based on the computation of the error function, is $\mathcal{O}(m \cdot n)$. The assignment step is the most computationally demanding, mainly due to the number of distance computations that need to be done. Algorithm 1 provides the pseudo-code of the K -means algorithm. Note that $\#\{a_i == j\}$ represents the count of data points assigned to cluster j . Here, a_i is the cluster assignment variable for the i -th data point (taking values in the set of integers $1, 2, \dots, K$). For coding purposes, it is easier to employ a_i instead of $\nu_{i,j}$ as defined previously. Note that $\nu_{i,j}$ is an indicator variable that takes the value of 1 if the i -th instance belongs to the j -th cluster, and 0 otherwise, while a_i is an integer encoding to which cluster sample $\mathbf{x}_i$ is assigned to.

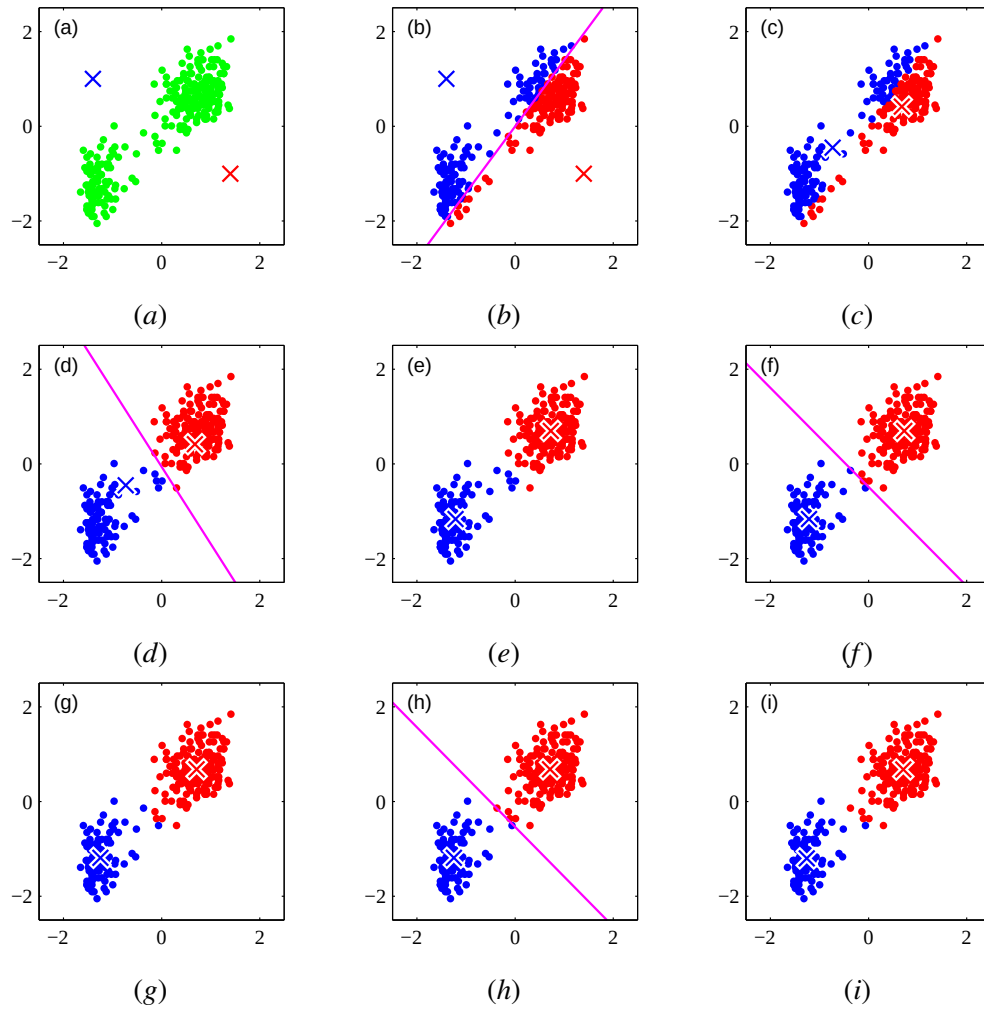


Figure 5: Illustration of the k-means algorithm (from (Bishop and Nasrabadi, 2006)). (a) Green points denote the data set in a two-dimensional Euclidean space. The initial choices for μ_1 and μ_2 are shown by red and blue crosses, respectively. (b) In the initial assignment step (called E-step), each data point is assigned either to the red cluster or to the blue cluster, according to which center it is closer to. This is equivalent to classifying points according to which side of the perpendicular bisector of the centers points are sitting. The bisector is shown by a magenta line. (c) In the next step (called M-step), each center is re-computed as the mean of all the points assigned to it. (d-i) Show successive runs of E and M steps until convergence.

Algorithm 1: k-Means algorithm

input : m data points, $\mathcal{X} = \{x_1, x_2, \dots, x_m\}$ with $x_i \in \mathbb{R}^n$.
output: the cluster centers $\mathcal{C} = \{\mu_1, \mu_2, \dots, \mu_K\}$ with $\mu_i \in \mathbb{R}^n$;
cluster assignments $\{a_1, a_2, \dots, a_m\}$, where $a_i \in \{1, 2, \dots, K\}$

Initialization;
Randomly pick K samples from $\mathcal{X}$ to initialize μ_i . Alternatively, initialize μ_i randomly (e.g., using a uniform distribution);

repeat
 forall $x_i \in \mathcal{X}$ **do**
 $d_{i,j}^2 = \|x_i - \mu_j\|^2$; /* compute the squared distance between x_i
 and each center μ_j */
 $a_i \leftarrow \arg \min_j d_{i,j}^2$; /* find the closest center to x_i and assign
 x_i to it */
 end
 forall $\mu_j \in \mathcal{C}$ **do**
 $\mu_j = \frac{1}{\#\{a_i=j\}} \sum_{i;\{a_i=j\}} x_i$; /* compute the center of each
 cluster based on the assignments a_i */
 end
until converged;

2.3. Initialization

It is widely observed that the performance of the K-means algorithm is highly dependent on the initialization stage (in terms of the quality of the solution and the running time). A poor initialization, for instance, could lead to an exponential running time⁵ in the worst-case scenario.

Ideally, the selected seeding/initialization strategy should deal with problems such as outliers. The standard initialization procedure consists of performing several re-initializations and keeping the set of centroids with the smallest error. The initial set of centroids here is randomly chosen from the dataset. The intuition behind this approach is that, by choosing the centroids uniformly at random, we are more likely to choose a point near an optimal cluster center since such points tend to be where the highest-density regions are located. Besides the fact that computing the error of each set of centroids is $\mathcal{O}(m \cdot K \cdot n)$ (due to the assignment step), the main disadvantage of this approach is that there is no guarantee that two, or more, of the selected seeds, will not be near the center of the same cluster, especially when dealing with unbalanced clusters.

More recently, simple probabilistic-based seeding techniques have been developed and, due to their simplicity and strong theoretical guarantees, they have become quite popular. Among these, the most popular one is the *K-means++* algorithm proposed by Arthur and Vassilvitskii ([Arthur and Vassilvitskii, 2007](#)). *K-means++* selects only the first centroid uniformly at random from the dataset. Each subsequent initial centroid is chosen with a probability proportional to the distance

⁵A function $f(n)$ is exponential if it has the form $a \times b^n$, where a and b are some constants. For example, 2^n , is an exponential function. A program or a function that has exponential running time is bad news because such programs run extremely slowly! Suppose the running time of a function is 2^n . Further, suppose that each primitive operation can be executed in one microsecond (i.e., 10^{-6} seconds). Then solving the problem for $n = 100$ will take $10^{-6} \times 2^{100}$ seconds. This equals 1.26765×10^{24} seconds, which is more than 10,000 trillion years, more than the lifetime of the universe by several orders of magnitude.

with respect to the previously selected set of centroids. The key idea of this cluster initialization technique is to preserve the diversity of seeds while being robust to outliers. The main drawback of this approach is its sequential nature, which hinders its parallelization. Also, the algorithm requires K full scans of the entire dataset, which leads to a complexity of $\mathcal{O}(m \cdot K \cdot n)$ just for initialization.

2.4. Determining the number of clusters

You may ask yourself *how can I determine the number of clusters* (i.e., the parameter K) for the *k-means algorithm*? The elbow method⁶ is a heuristic approach used in determining the number of clusters in a data set. The method consists of plotting the k-means distortion (aka loss as defined in Equation (2)) as a function of the number of clusters and picking the elbow of the curve as the number of clusters to use. An elbow or knee of a curve is a point where the curve visibly bends, specifically from a high slope to a low slope (flat or close to flat), or in the other direction (see Figure 6 for an example).

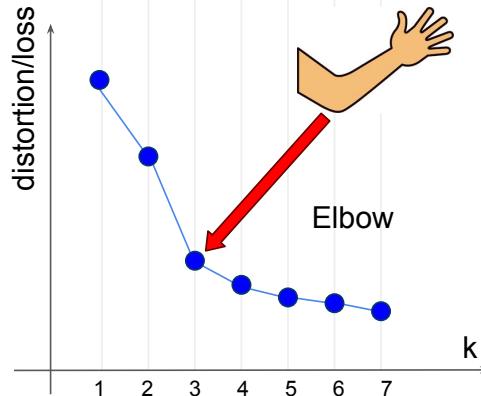


Figure 6: The optimal number of clusters can be determined by identifying the elbow of distortion/loss vs. k .

2.5. Convergence †⁷

In computer science, when you see an iterative algorithm, you should immediately ask yourself, *does this algorithm converge*? If yes, how? By how, I mean understanding the speed (i.e., rate) of convergence (does the algorithm take ten minutes to generate its outputs or ten decades) if some constraints need to be satisfied for successful convergence and etc. In this part, we are not interested in how questions, but it is going to be very educational to study the convergence of the K-means algorithm. Let us rewrite the loss of K-means as

$$\mathcal{L}(\mathcal{C}, \{y_i\}) = \sum_{i=1}^n \|x_i - \mu_{y_i}\|^2. \quad (5)$$

⁶According to Wikipedia, the elbow method can be traced to speculation by Robert L. Thorndike in 1953 [Thorndike \(1953\)](#)

⁷This is outside the scope of our course and is here just for curious minds. Feel free to skip as it will not be on exam.

Here, y_i denotes the assignment of x_i to its nearest center, i.e., $y_i = \arg \min_j \|x_i - \mu_j\|^2$. First convince yourself that we did not cheat and trick you and Equation (5) is exactly Equation (2), but re-branded to serve our purpose better here. Also, recall that $\mathcal{C} = \{\mu_1, \mu_2, \dots, \mu_K\}$.

First note that $\mathcal{L}(\mathcal{C}, \{y_i\}) \geq 0$ from (5). To prove the convergence, we show that the loss function is guaranteed to decrease monotonically in each iteration for the assignment step and for the updating step. As such, the algorithm will eventually converge when the loss function reaches its (local) minimum. I emphasize that **1.** we reduce the loss monotonically, **2.** the loss is lower-bounded and hence we cannot reduce it forever, **3.** if we see no progress in the algorithm, that is no change in assignments and cluster centers, we will stop. However, this does not mean that we have reached the global minimum of the algorithm as the loss is not convex.

Assignment step. Let's reiterate the setup during the assignment step. We have a set of cluster centers from a previous iteration (or in the beginning, from initialization), which we show with $\{\mu_j\}_{j=1}^K$. Consider a point x_i . Recall that the assignment step is run after recomputing the cluster centers. Let us show the assignment of x_i at the previous iteration of the algorithm by y_i . The current assignment of x_i is shown by y_i^* . That is $y_i^* = \arg \min_j \|x_i - \mu_j\|^2$. Note that with new centers, the old assignments shown by y_i are no-longer optimum. What we want to show is that the loss with previous assignments $\mathcal{L}(\mathcal{C}, \{y_i\})$ is bigger than the loss with current assignments $\mathcal{L}(\mathcal{C}, \{y_i^*\})$. This is quite easy to see as

$$\|x_i - \mu_{y_i}\|^2 \geq \|x_i - \mu_{y_i^*}\|^2.$$

This is because we choose y_i^* to minimize the distance. Now since the loss is simply a sum over *non-negative* terms like $\|x_i - \mu_{y_i^*}\|^2$ and for each term the inequality above holds, then we can conclude $\mathcal{L}(\mathcal{C}, \{y_i\}) \geq \mathcal{L}(\mathcal{C}, \{y_i^*\})$, and hence the assignment step reduces the loss (or better say cannot lead to an increase in the loss).

Update step. Now assume we have obtained our new assignments y_i^* . From this, we will update the cluster centers as

$$\mu_j^* = \frac{1}{\#\{y_i^* = j\}} \sum_{y_i^*=j} x_i.$$

We can now compare the loss before and after the update, i.e., $\mathcal{L}(\mathcal{C}, \{y_i^*\})$ and $\mathcal{L}(\mathcal{C}^*, \{y_i^*\})$. The loss for cluster j is obviously reduced as μ_j^* is indeed the minimizer of the sum of squared distances. Hence,

$$\sum_{y_i^*=j} \|x_i - \mu_j\|^2 \geq \sum_{y_i^*=j} \|x_i - \mu_j^*\|^2.$$

Since the total loss is the sum of terms in the form $\sum_j \sum_{y_i^*=j} \|x_i - \mu_j^*\|^2$, then we conclude that the updating step also monotonically reduces the total loss and hence the algorithm will converge.

2.6. K-Medoids Algorithm ^{†8}

Sometimes, we encounter situations where we want to group together objects that can't be averaged. One such example is when we're analyzing medical images, such as those produced by [Diffusion](#)

⁸This is outside the scope of our course and is here just for curious minds. Feel free to skip as it will not be on exam.

Tensor Imaging (DTI). Each pixel in a DTI image represents the physical properties of a water molecule, so simply averaging pixels may lead to results that are not physically possible! A variant of the K-means algorithm, known as *K-medoids*, can help us cluster in such cases. In K-medoids, the cluster centers are data items rather than averages and are called “medoids.” The k-medoids algorithm has a familiar form to k-means. We assume K , the number of cluster centers, is known. We initialize the cluster centers by choosing examples at random. We then repeatedly perform two procedures: **1.** assign each data point to the closest medoid, and **2.** choose the best medoid for each cluster by finding the data point that minimizes the sum of squared distances of points in the cluster to that medoid. This point can be found by simply searching all the points in the cluster.

Remark 1 *The k-means clustering algorithm is sensitive to outliers because extreme values easily influence the averaging operation. K-medoids, on the other hand, is more robust to noises and outliers. See Figure 7 for an illustration.*

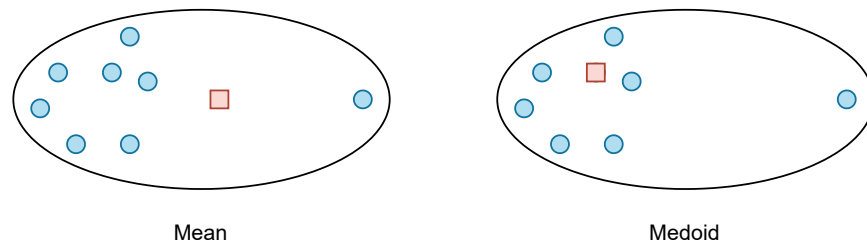


Figure 7: Mean vs. medoid in 2-D space. In both panels, the group of blue circles forms a cluster, with the rightmost point being an outlier. The red square represents the cluster centroid found by mean or medoid.

3. Gaussian Mixture Models (GMMs)

Recall the pdf of a univariate Gaussian distribution as

$$\mathcal{N}(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(x - \mu)^2\right). \quad (6)$$

The parameter μ is the mean of the distribution (and also its median and mode for the Gaussian distribution), while the parameter σ is its standard deviation (see Figure 8 for an illustration).

The extension of the univariate Gaussian to n -dimensional spaces reads as

$$\mathcal{N}(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{\sqrt{(2\pi)^n \det(\boldsymbol{\Sigma})}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right). \quad (7)$$

The way we need to think about a pdf, in this case, a multivariate, is to think about random vectors. Here, $\mathbf{x} \in \mathbb{R}^n$ is our random vector, meaning that each element of it is a random variable. The vector $\boldsymbol{\mu} \in \mathbb{R}^n$ is the mean of the distribution and $\boldsymbol{\Sigma} \in \mathbb{R}^{n \times n}$ is the covariance matrix of the

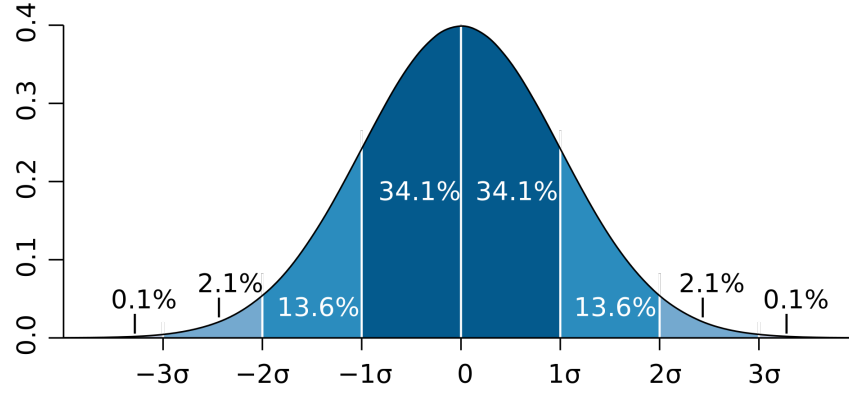


Figure 8: An illustration of the Gaussian distribution. For the normal distribution, the values less than one standard deviation away from the mean account for 68.8% of the set, while two standard deviations from the mean account for 95.5%; and three standard deviations account for 99.7% (figure courtesy: [Wikipedia](#)).

distribution.⁹ The inverse of a covariance matrix is known as the **precision** matrix, also sometimes called the **concentration** matrix.

Similar to the 1D case, you can estimate the parameters of a multivariate Gaussian distribution from its samples. Let $\mathbb{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$ be a set of samples from a Gaussian distribution with the mean and covariance $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$. The maximum likelihood estimation of $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ using $\mathbb{X}$ reads as,

$$\hat{\boldsymbol{\mu}} = \frac{1}{m} \sum_{i=1}^m \mathbf{x}_i, \quad (8)$$

$$\hat{\boldsymbol{\Sigma}} = \frac{1}{m} \sum_{i=1}^m (\mathbf{x}_i - \hat{\boldsymbol{\mu}})(\mathbf{x}_i - \hat{\boldsymbol{\mu}})^\top. \quad (9)$$

A covariance matrix is a square matrix giving the covariance between each pair of elements of a random vector. Covariance matrices are symmetric and positive semi-definite, and their main diagonal contains variances. Informally, the covariance between two random variables shows how linearly the random variables change together. The covariance matrix has the following form:

$$\boldsymbol{\Sigma} = \begin{bmatrix} \sigma_1^2 & \sigma_{1,2} & \cdots & \sigma_{1,n} \\ \sigma_{2,1} & \sigma_2^2 & \cdots & \sigma_{2,n} \\ \vdots & \ddots & \ddots & \vdots \\ \sigma_{n,1} & \sigma_{n,2} & \cdots & \sigma_n^2 \end{bmatrix}. \quad (10)$$

In (10), σ_i^2 is the variance of the i -th random variable (element i -th of the random vector which is normal) and $\sigma_{i,j} = \sigma_{j,i}$ is the covariance between the i -th and j -th random variables. In the case of

⁹To be precise, $\boldsymbol{\Sigma}$ is a symmetric positive definite matrix. Geometrically, the set of $n \times n$ covariance matrices forms the cone of symmetric positive definite (SPD) matrices. The dimensionality of the cone is $n(n+1)/2$.

a Gaussian distribution, if $\sigma_{i,j} = 0$, then the two random variables are independent. Figure 9 shows two examples of the bivariate Gaussian distribution (*i.e.*, when $n = 2$).

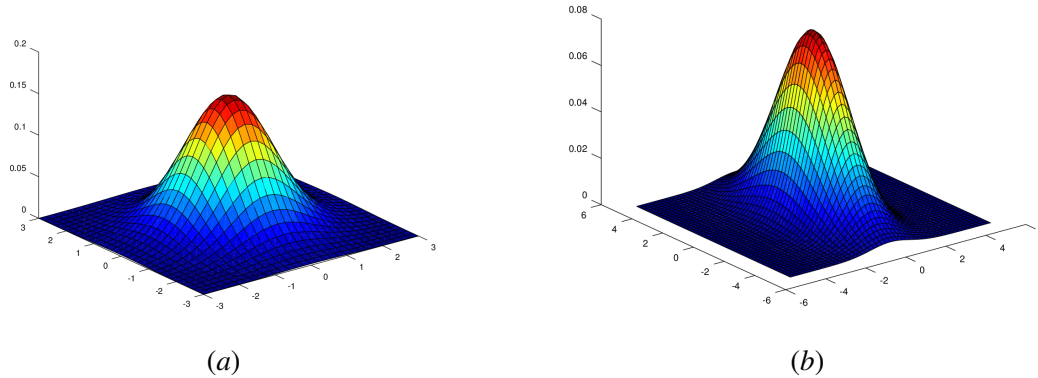


Figure 9: An illustration of the bivariate Gaussian (normal) distribution. The covariance matrix of a Gaussian distribution shows the correlation between variables and the strength (variance) of each random variable.

In Equation (9), I provide a concise way of computing Σ . To better understand the off-diagonal elements of Σ , note that

$$\sigma_{r,s} = \frac{1}{m} \sum_{i=1}^m (x_i[r] - \hat{\mu}[r]) (x_i[s] - \hat{\mu}[s]) . \quad (11)$$

3.1. From K-Means to GMMs

While we can approach the clustering problem as fitting distributions to data, we will focus on a more intuitive understanding of GMMs here. K-Means is a fantastic algorithm, works in many situations like a charm, and is perhaps our very first choice in practice. However, the algorithm comes with several shortcomings, including

- In some, if not many cases, a data point belongs to more than one cluster. The K-means algorithm is not able to model such scenarios. For example, an image may contain several objects, say a man riding a horse. Assigning the image to just one cluster, either the cluster for persons or horses, is not ideal.
- K-means can model compact and dense clusters well. For elongated clusters, the algorithm fails to capture the structure of data. This is illustrated conceptually in Figure 10.
- K-Means clustering does not encode any notion of cluster size. You can have a cluster with merely two samples and another cluster from one million samples. Obviously, these two clusters should be treated differently and in particular, weighted accordingly. However, K-Means is blind to the size of clusters.

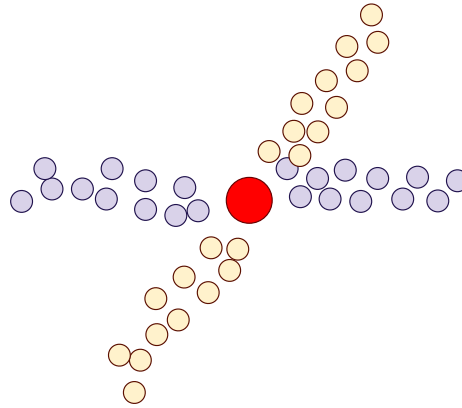


Figure 10: K-Means fails to capture the structure of data for elongated clusters. For both data structures shown above, the K-Means finds the same cluster center. In many applications, we only keep the cluster centers for inference and as such, we fail to confidently assign new samples to their corresponding clusters as the structure is not encoded by K-Means.

To address the aforementioned drawbacks, we use GMMs. Let's go back to our beloved Gaussian distribution in n -dimensional spaces. In Figure 11, contour surfaces of a bivariate Gaussian distribution are shown for several different covariance matrices.

We note that an n -dimensional Gaussian distribution gives heaps of flexibility in the modeling data structure. It is rich and can be used to model elongated clusters. Also, similar to the clusters in K-Means, it also uses the center of the cluster (*i.e.*, μ) along with the covariance matrix for modeling. This property of the Gaussian distribution will serve us as a means to model the structure of clusters way better than what K-Means can do. Furthermore, note that a Gaussian distribution covers the whole $\mathbb{R}^n$. Even very far away points still have some likelihood with respect to a Gaussian distribution. We will use this property to devise a soft assignment mechanism for clustering. Hence an input can be assigned to multiple clusters depending on its similarity to the clusters (which is defined by the Gaussian distribution). The very last piece of our puzzle is a way to encode the size of clusters. This, we can do by defining a weight for each component. A higher weight means that the corresponding component contributes more. All these properties and design choices are encapsulated in a GMM, so drum roll, please.

Gaussian Mixture Models

GMMs are a type of probabilistic model where each cluster is represented by a Gaussian distribution. The GMM is a weighted sum of K Gaussian densities, defined as:

$$p(\mathbf{x}) = \sum_{j=1}^K w_j \phi_j(\mathbf{x}) , \quad (12)$$

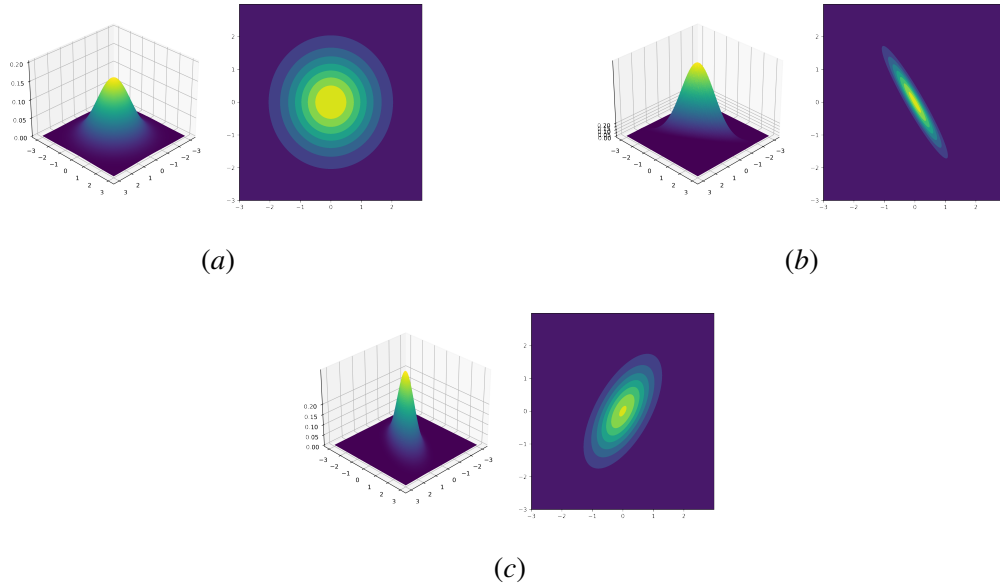


Figure 11: A bivariate Gaussian distribution can model various elliptical surfaces. Here, three examples are shown. Please note that stupid Matplotlib does not adjust the camera angle automatically according to what I want from 3D plots (am I too demanding?). I was too lazy to figure out how to correct this (and also ChatGPT was not released when I was writing this part) so please rotate the 3D plots in your head or simply focus on the contour plots! Also, I forgot to add the color bar but brighter regions show higher similarities (which is obvious from the 3D plots).

Here, K is the number of clusters or modes, and w_j represents the weight of the j -th cluster. The set of weights $\{w_1, w_2, \dots, w_K\}$ should satisfy the following two properties

$$w_j \geq 0 \quad (13)$$

$$\sum_{j=1}^K w_j = 1. \quad (14)$$

If a cluster contains more data points, we expect it to be more important and hence to have a higher weight. The term $\phi_j(\mathbf{x})$ in Equation (12) denotes the Gaussian distribution of the j -th cluster with mean $\boldsymbol{\mu}_j$ and covariance $\boldsymbol{\Sigma}_j$ as

$$\phi_j(\mathbf{x}) = \frac{1}{\sqrt{(2\pi)^n \det(\boldsymbol{\Sigma}_j)}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_j)^\top \boldsymbol{\Sigma}_j^{-1}(\mathbf{x} - \boldsymbol{\mu}_j)\right). \quad (15)$$

Let us understand Equation (12) along with its requirements (*i.e.*, Equations (13) and (14)) better. It is important to verify that Equation (12) is indeed a probability density function so that we can use the GMM to make probabilistic predictions about data points. To show that Equation (12) is a valid pdf, we need to ensure that $0 \leq p(\mathbf{x}) \leq 1$; $\forall \mathbf{x}$ and $\int p(\mathbf{x}) d\mathbf{x} = 1$. The first requirement can be easily verified (I leave this as an exercise to you), and to show the second requirement, we can write:

$$\begin{aligned}
\int_{\mathbf{x}} p(\mathbf{x}) d\mathbf{x} &= \int_{\mathbf{x}} \sum_{j=1}^K w_j \phi_j(\mathbf{x}) d\mathbf{x} \\
&\stackrel{(i)}{=} \sum_{j=1}^K w_j \int_{\mathbf{x}} \phi_j(\mathbf{x}) d\mathbf{x} \\
&\stackrel{(ii)}{=} \sum_{j=1}^K w_j = 1 .
\end{aligned}$$

Here, in (i), we simply swap the order of the sum and integral (note that w_j is a fixed weight and not dependent on $\mathbf{x}$ and hence coming out of the integral) and in (ii), we use the fact that $\phi_j(\mathbf{x})$ is a Gaussian distribution and hence the integral over it is 1 (note that $\sum_{j=1}^K w_j = 1$ is by the construction of GMM).

A GMM model is capable of assigning soft memberships to data points. Specifically, the probability of a data point $\mathbf{x}$ belonging to the k -th cluster can be computed as

$$\gamma_k(\mathbf{x}) = \frac{w_k \phi_k(\mathbf{x})}{\sum_{j=1}^K w_j \phi_j(\mathbf{x})} . \quad (16)$$

We note that $p(\mathbf{x}) = \sum_{j=1}^K w_j \phi_j(\mathbf{x})$ is the posterior probability of the model. The quantity $\gamma_k(\mathbf{x})$ is called the responsibility. If a point needs to be assigned to only one cluster, this can be done by selecting the cluster with the highest responsibility of cluster k given $\mathbf{x}$ or simply the responsibility.

Model Fitting. To use the GMM for clustering, one needs to estimate the parameters $\Omega = \{w_j, \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j\}_{j=1}^K$ of the model. This involves finding the values of the parameters that maximize the likelihood of the observed data points. To be more specific, assume you are given a set of points $\mathbb{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$. We assume these points are sampled independently from a distribution that we hope to approximate with a GMM. The likelihood of $\mathbb{X}$ being generated by the GMM can be written as

$$\begin{aligned}
\mathbb{P}(\mathbb{X}|\Omega) &\stackrel{(i)}{=} p(\mathbf{x}_1) \times p(\mathbf{x}_2) \times p(\mathbf{x}_3) \times \dots \times p(\mathbf{x}_m) = \prod_{i=1}^m p(\mathbf{x}_i) \\
\Rightarrow \log(\mathbb{P}(\mathbb{X}|\Omega)) &\stackrel{(ii)}{=} \sum_{i=1}^m \log(p(\mathbf{x}_i)) = \boxed{\sum_{i=1}^m \log\left(\sum_{j=1}^K w_j \phi_j(\mathbf{x}_i)\right)} . \quad (17)
\end{aligned}$$

Above, (i) follows from the independence of samples, and (ii) is the property of the log function. If you try to maximize Equation (17) in the hope of finding the best parameters to describe your data samples, you will see that the resulting equations do not admit a closed-form solution. The Expectation-Maximization (EM) algorithm (Dempster et al., 1977) is a popular method for estimating the parameters of a clustering model. The algorithm consists of two steps: the E-step, where the posterior probabilities of the data points belonging to each cluster are estimated, and the M-step, where the model parameters are updated by maximizing the expected complete-data log-likelihood. This process iterates until convergence is reached, resulting in a set of parameters that can be used to

cluster new data points. During the E-step, the algorithm computes the responsibilities $\{\gamma_k(\mathbf{x}_i)\}_{i=1}^m$ using the current estimation of the parameters. In the M-step, the computed responsibilities are used to update the parameters of the model. Although the derivation of the algorithm is not difficult, it can be hairy. I leave it to an interested reader to follow the derivation from a textbook such as [Bishop and Nasrabadi \(2006\)](#) and provide the updating rules instead.

$$\boldsymbol{\mu}_k^{\text{new}} = \frac{1}{N_k} \sum_{i=1}^m \gamma_k(\mathbf{x}_i) \mathbf{x}_i . \quad (18)$$

$$\boldsymbol{\Sigma}_k^{\text{new}} = \frac{1}{N_j} \sum_{i=1}^m \gamma_k(\mathbf{x}_i) (\mathbf{x}_i - \boldsymbol{\mu}_k^{\text{new}}) (\mathbf{x}_i - \boldsymbol{\mu}_k^{\text{new}})^\top . \quad (19)$$

$$w_k^{\text{new}} = \frac{N_k}{m} . \quad (20)$$

In Equations (18), (19) and (20), N_k is defined as

$$N_k = \sum_{i=1}^m \gamma_k(\mathbf{x}_i) . \quad (21)$$

You can interpret N_k as the effective number of points assigned to cluster k . To see this, consider the responsibilities $\gamma_k(\mathbf{x})$, which represent the posterior probability that a data point $\mathbf{x}$ belongs to cluster k given the current estimate of the model parameters. If we assume that the responsibilities take binary values, *i.e.*, $\gamma_k(\mathbf{x}) = 1$ for only one of the clusters and 0 for the rest, then it is obvious that N_k counts the number of points assigned to cluster k . In practice, the responsibilities take continuous values between 0 and 1, which reflect the degree of membership of each data point to the different clusters. For example, if $\gamma_k(\mathbf{x}) = 0.8$, we can interpret this as $\mathbf{x}$ having an 80% likelihood of belonging to cluster k . In this case, N_k is a weighted sum of the responsibilities, where the weights reflect the degree of membership of each data point to cluster k . It is worth noting that $0 \leq \gamma_k(\mathbf{x}_i) \leq 1$ and $\sum_{j=1}^K \gamma_j(\mathbf{x}) = 1$ for all data points $\mathbf{x}$.

Remark 2 *I keep using this weird term “Posterior” without being explicit. So you may ask, what does this even mean? In the context of GMMs, the “posterior probability” refers to the probability of a sample belonging to a particular cluster, given the model. In contrast, the “prior probability” is the probability of a sample coming from that cluster before observing any data, which is simply the weight assigned to each cluster k (*i.e.*, w_k). To make this more concrete, consider a scenario where we have a GMM and we want to determine the probability of a sample $\mathbf{x}$ coming from cluster k . We can guess the probability without even looking at the data by checking the weight assigned to cluster k . However, if we want a more detailed answer, we can compute the posterior probability $\gamma_k(\mathbf{x})$, which takes into account the likelihood of the sample coming from each cluster as well as the prior probability. It is worth noting that $\gamma_k(\mathbf{x})$ is a function of both the data and the parameters of the model, and provides a more informative way of describing the membership of a data point in each cluster.*

Compared to K-means, GMMs can model elongated clusters well, as they can capture the covariance structure of the data. Moreover, GMMs can encode the size of each cluster by weighting the contribution of each cluster based on its prior probability. Finally, GMMs can also model overlapping clusters, where a data point may belong to multiple clusters with different probabilities.

Algorithm 2 presents the pseudo-code for training a GMM. The term $\mathcal{L}(\mathbb{X}, \Omega^{(\text{new})})$ represents the likelihood of the model and we expect it to increase after each iteration of the algorithm and can be used to determine the convergence of the algorithm.

Algorithm 3: EM algorithm for training GMMs

Input: Data set $\mathbb{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_m\}$ and initial parameter values $\Omega^{(\text{old})}$

Output: Parameter values $\Omega^{(\text{new})}$ that maximize the likelihood of the data set

while *not converged* **do**

E-Step;

 For each data point $\mathbf{x}_i$, calculate the responsibilities $\gamma_k(\mathbf{x}_i)$ for cluster $k \in \{1, 2, \dots, K\}$ according to:

$$\gamma_k(\mathbf{x}_i) = \frac{w_k \phi_k(\mathbf{x}_i)}{\sum_{j=1}^K w_j \phi_j(\mathbf{x}_i)}$$

M-Step;

 Update the parameters as follows:

- Compute the effective number of points for all clusters $k \in \{1, 2, \dots, K\}$

$$N_k = \sum_{i=1}^m \gamma_k(\mathbf{x}_i)$$

- Update the mixing coefficients w_k :

$$w_k = \frac{1}{m} N_k$$

- Update the means $\boldsymbol{\mu}_k$:

$$\boldsymbol{\mu}_k^{\text{new}} = \frac{1}{N_k} \sum_{i=1}^m \gamma_k(\mathbf{x}_i) \mathbf{x}_i$$

- Update the covariance matrices $\boldsymbol{\Sigma}_k$:

$$\boldsymbol{\Sigma}_k^{\text{new}} = \frac{1}{N_k} \sum_{i=1}^m \gamma_k(\mathbf{x}_i) (\mathbf{x}_i - \boldsymbol{\mu}_k^{\text{new}}) (\mathbf{x}_i - \boldsymbol{\mu}_k^{\text{new}})^\top$$

- Calculate the log-likelihood of the data set under the new parameter values:

$$\mathcal{L}(\mathbb{X}, \Omega^{(\text{new})}) = \sum_{n=1}^m \log \left(\sum_{j=1}^K w_j \phi_j(\mathbf{x}_i) \right)$$

 Update the parameter values $\Omega^{(\text{old})} \leftarrow \Omega^{(\text{new})}$

end

I shamelessly provide the example from the book (Bishop and Nasrabadi, 2006) here. In Figure 12, we consider a GMM with two components and initialize the centers using the same values as for the K-means algorithm discussed previously in Figure 5. Furthermore, the precision matrices

(inverse covariance matrices) are initialized as identity matrices. Plot Figure 12(a) shows the data points in green, together with the initial configuration of the GMM. Plot Figure 12(b) shows the result of the initial assignment step. Note that in this plot, each data point is depicted using a proportion of blue ink equal to the likelihood of being from the blue component and a corresponding proportion of red ink by the likelihood of the red component. Thus, points that have a significant probability of belonging to the blue/red component will be seen as blue/red. Purple points are the ones that can be with either cluster (*i.e.*, the likelihood is roughly 50/50). After the first M step, where we update the parameters of the model, is shown in plot Figure 12(c). There, we notice that the mean of the blue Gaussian has moved to the centre of mass of the blue ink. Similarly, the covariance of the blue Gaussian is set equal to the covariance of the blue ink. Analogous results hold for the red component. Note that the mean and covariance of the components are defined by the likelihood of all samples in the data set according to their probabilities. Plots Figure 12(d), Figure 12(e), and Figure 12(f) show the results after 2, 5, and 20 full iterations of the algorithm, *i.e.*, E-step (or assignment) + M-step (or update), respectively. In plot Figure 12(f), the algorithm is close to convergence.

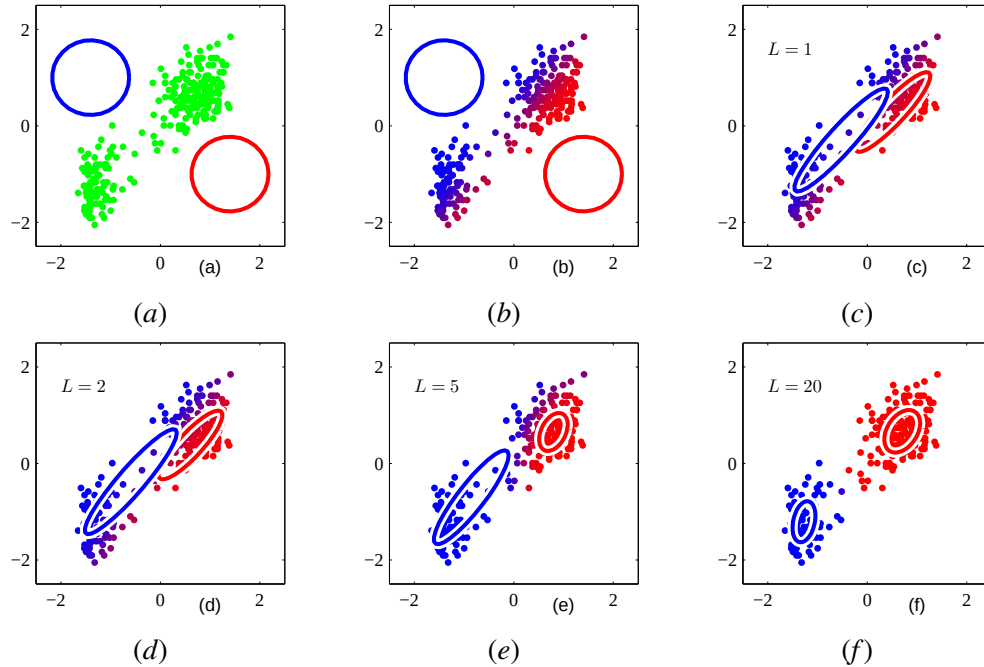


Figure 12: Illustration of the GMM algorithm for clustering (from (Bishop and Nasrabadi, 2006)). See the text for details.

Remark 3 (The Log-Sum-Exp Trick) *The computation in the form of Equation (22) is a familiar occurrence in many ML techniques, in the case of GMMs in computing the responsibilities $\gamma_k(\mathbf{x})$.*

$$\log \left(\sum_{i=1}^m \exp(d_i) \right). \quad (22)$$

Naively computing Equation (22) can suffer from numerical instabilities, especially when the values of d_i are large. One way to address this issue is to use the log-sum-exp trick, which is based on the following identity¹⁰:

$$\log \left(\sum_{i=1}^m \exp(d_i) \right) = a + \log \left(\sum_{i=1}^m \exp(d_i - a) \right). \quad (23)$$

In practice, a is chosen as the maximum of $\{d_1, d_2, \dots, d_m\}$. By subtracting such a (i.e., $a = \max d_i$) from each d_i , we ensure that the largest term in the sum is 1, which helps to avoid numerical issues.

4. Beyond K-means and GMMs

One of the major difficulties in using k-means (or, for that matter GMMs) is the reliance on an a priori k . Mean shift [Comaniciu and Meer \(2002\)](#) algorithm addresses that issue for us. To understand the algorithm, we need to change our perspective and think about the distribution of our data. Recall that in K-Means; we assume that there are some “spherical clusters” in the feature space that we try to separate (i.e., segment). If we think of data as a probability density (each data point has a likelihood of happening), the clusters can be thought of as the **modes**¹¹ of the distribution. If we could find those modes (i.e., local maxima), we would be able to cluster data.

Finding Modes of a Distribution. Recall that a GMM combines a set of Gaussian distributions to model complex distributions (see Figure 13 that I grabbed from this beautiful [animated GIF](#) for an example). Think again about how a GMM can be used for image segmentation. You fit a GMM to your data, and then each mode of the GMM defines a group of pixels that form a cluster. Now you can ask yourself a question; *if each cluster is equivalent to a mode of distribution (unknown to us), can we drop the assumption that the distribution is made of local Gaussian functions and directly find the modes?* The mean shift algorithm is the answer to this question.

The algorithm is very simple. Assume a set of points is given $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$, $\mathbf{x}_i \in \mathbb{R}^n$. Start with an initial estimate of the mode shown by $\mathbf{m}^{(0)}$. Let $k : \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}$ be a kernel function¹² such as an RBF function¹³ $k(\mathbf{x}, \mathbf{z}) = \exp(-\sigma \|\mathbf{x} - \mathbf{z}\|^2)$ and consider the following quantity

$$\mathbf{m}^{(t+1)} \leftarrow \frac{\sum_{\mathbf{x}_i \in \mathfrak{N}(\mathbf{m}^{(t)})} k(\mathbf{x}_i, \mathbf{m}^{(t)}) \mathbf{x}_i}{\sum_{\mathbf{x}_i \in \mathfrak{N}(\mathbf{m}^{(t)})} k(\mathbf{x}_i, \mathbf{m}^{(t)})}. \quad (24)$$

Here, $\mathfrak{N}(\mathbf{m}^{(t)})$ denotes the set of neighbors of $\mathbf{m}^{(t)}$. Let’s take a moment to understand Equation (24). First, note that a kernel function is a notion of similarity between two points; the closer they are, the more similar they are (hence $k(\mathbf{x}, \mathbf{m})$ increases if $\mathbf{x}$ is closer to $\mathbf{m}$). So, in essence,

¹⁰Recall that $\log(ab) = \log(a) + \log(b)$.

¹¹The mode of a probability distribution is the value (or values) in support of the distribution that corresponds to the highest peak (or peaks) of its PDF. In other words, mode represents a distribution’s most frequent or most likely outcome(s).

¹²The notation $k : \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}$ tells you that k is a function that accepts two vectors in $\mathbb{R}^n$ (this is denoted by $\mathbb{R}^n \times \mathbb{R}^n$) and generates a value in $\mathbb{R}$ (i.e., the image of the function is $\mathbb{R}$).

¹³A Radial Basis Function (RBF) is a real-valued function whose value depends only on the distance between its arguments. In other words, $k(\mathbf{x}, \mathbf{z})$ is defined as a function of $\|\mathbf{x} - \mathbf{z}\|^2$, hence if $\mathbf{z}$ gets farther and farther away from $\mathbf{x}$, $k(\mathbf{x}, \mathbf{z})$ decreases.

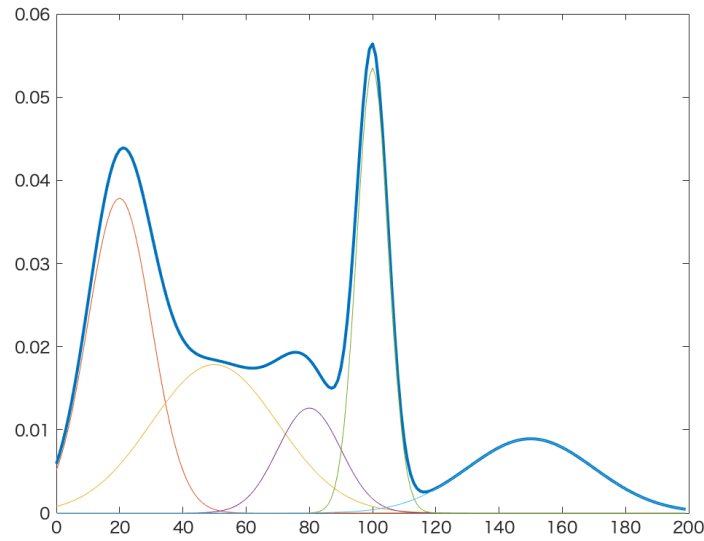


Figure 13: A complex distribution (in blue) can be decomposed as a mixture of Gaussian distributions.

Equation (24) defines a weighted averaging over neighbors of $\mathbf{m}$. Now, consider the difference between two consecutive values of $\mathbf{m}$, i.e., $\vec{\mathbf{m}} := \mathbf{m}^{(t+1)} - \mathbf{m}^{(t)}$. This is simply called the **mean shift** vector! The mean-shift algorithm uses $\mathbf{m}^{(t)}$, and repeats until $\mathbf{m}^{(t)}$ converges to find the mode of distribution (unknown distribution)!

To intuitively understand this better, consider the distribution shown in Figure 14. Imagine we start at x_1 , a point corresponding to the pdf at the red point. If you imagine an interval in the form $[x_1 - \delta, x_1 + \delta]$, you can conclude that the average of points falling into this interval is going to be on the right of x_1 . This is because we have a higher pdf on the right, so we will get more points on that side. If we move toward the mean inside the interval, we will climb up the distribution. You can make a similar argument about x_2 . However, here we will move to the left of the point, but still this leads to climbing up the pdf. If we start at x_1 and iteratively pick an interval around our current estimation and move there, we will end up in x^* , the mode of the distribution. Only there, we cannot find a denser location, and hence we stop. Now, think about where you may end up if you start from x_2 ¹⁴?

To gain an intuitive understanding of the mean-shift algorithm, consider the 1D bimodal distribution shown in Figure 14. Suppose we start at point x_1 with the corresponding pdf value shown by the red circle. If we pick an interval¹⁵ around x_1 and compute the mean (i.e., average) of points within that interval, we will get a value that is on the right of x_1 . This is because the probability density is higher on the right side, so we will get more points on that side. If we move towards the mean inside the interval, we will climb up the distribution. Similarly, if we start at x_2 , we will move

¹⁴In this case, we will find the second mode of the distribution.

¹⁵This is known as **flat kernel**. To be specific, a flat kernel is a non-parametric kernel function that assigns equal weight to all points within a given radius, regardless of their distance from the center point. It is also known as a uniform kernel or rectangular kernel.

to the left of that point if we move towards the mean in an interval around x_2 , hence again climbing up the distribution. Starting from x_1 and by iteratively picking an interval around the current estimate and moving towards the mean inside that interval, we will eventually end up at the mode of the distribution, denoted by x^* . At the mode, we cannot find a denser location, so we stop. If we start at x_2 , we will end up at a different mode, as there are two peaks in the distribution.

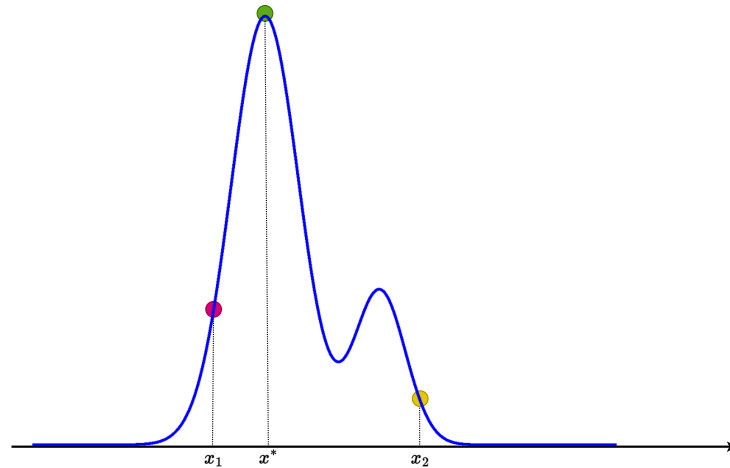


Figure 14: Illustration of how the mean-shift algorithm can be used to find the mode of a 1D bimodal distribution. Starting from an initial point (e.g., x_1) and using an interval of a certain size as the kernel function, the algorithm iteratively shifts the estimate of the mean towards denser regions of the distribution until convergence to the mode is achieved.

Remark 4 *The discussion above can be readily generalized to n -dimensional spaces as well. See an example in $\mathbb{R}^2$ in Figure 15.*

Remark 5 *In our discussion above, we used a hard constraint in the form of an interval to include points in computing the new estimate of the mean. However, the use of an RBF kernel (or any smooth kernel) allows for all points to be considered in obtaining the mean rather than being restricted by a hard constraint.*

5. Some Applications

5.1. Image Segmentation and Compression

As an application of clustering, we consider the problem of image segmentation. The goal of segmentation is to partition an image into disjoint regions, where each region is reasonably homogeneous in its visual appearance. Regions ideally should correspond to objects or parts of objects. The term image segmentation is used for a wide range of activities, although techniques may differ. That said, the motivation for all these activities is the same: obtain a compact representation of what is helpful in the image [Forsyth and Ponce \(2011\)](#).

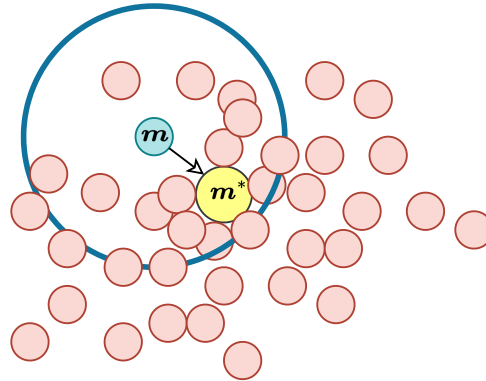


Figure 15: The figure illustrates the mean shift algorithm in 2D space using a flat kernel. Red circles represent individual data points, while the light blue circle m represents the previous mean estimate, and the yellow circle m^* represents the current estimate obtained as the weighted average of points within a radius around the previous mean (shown by the blue circle).

Recall that each pixel in an image is a point in a 3-dimensional space comprising the intensities of the red, blue, and green channels. Consider the K-means algorithm. The K-means simply treats each pixel in the image as a separate data point¹⁶, and hence can split an image into disjoint regions, where pixels in each region are similar according to their RGB values. We cautiously say pixels are similar according to their RGB values and not color because a slight change in any channel of the RGB color space could lead to a tangible change in the color we perceive. In other words, while pixels might be similar or close in the RGB space, they may exhibit different colors to our eyes. One could opt for other color spaces such as HSV to alleviate this. Regardless, the discussion on using K-means for segmentation holds.

Assume you have applied the K-means algorithm to the pixels of an image. As a result, you will have a set of cluster centers $\{\mu_1, \mu_2, \dots, \mu_K\}$, representing the RGB values of the image. Now, you can re-draw the image by replacing the RGB value of each pixel with its closest cluster center. That is, you can consider $\mu_i \in \mathbb{R}^3$ encoding an (R, G, B) intensity triplet. To get a feeling about this, consider the tiger image in Figure 16 and the result of applying K-means for various values of K . We see that for a given value of K , the algorithm is representing the image using a palette of only K colours. This use of K-means is not a particularly sophisticated approach to image segmentation¹⁷. This is because the algorithm does not take into account the spatial information in any sense, and we know how important such information is in images. On that note, we emphasize that the image segmentation problem is in general extremely difficult and remains the subject of active research and is introduced here simply to illustrate the behaviour of the K-means algorithm.

¹⁶Note that strictly speaking, the space is not Euclidean because the channel intensities are discrete, and bounded in the interval $[0, 1]$ (or for that matter in $\{0, 1, \dots, 255\}$). Nevertheless, we can apply the K-means algorithm without difficulty, as the goal is to group pixels together. Interestingly, cluster centers obtained by the K-means are bounded in $[0, 1]$ (why?).

¹⁷Though you will do it in the lab just for the fun of it and of course its valuable learning-prospect!

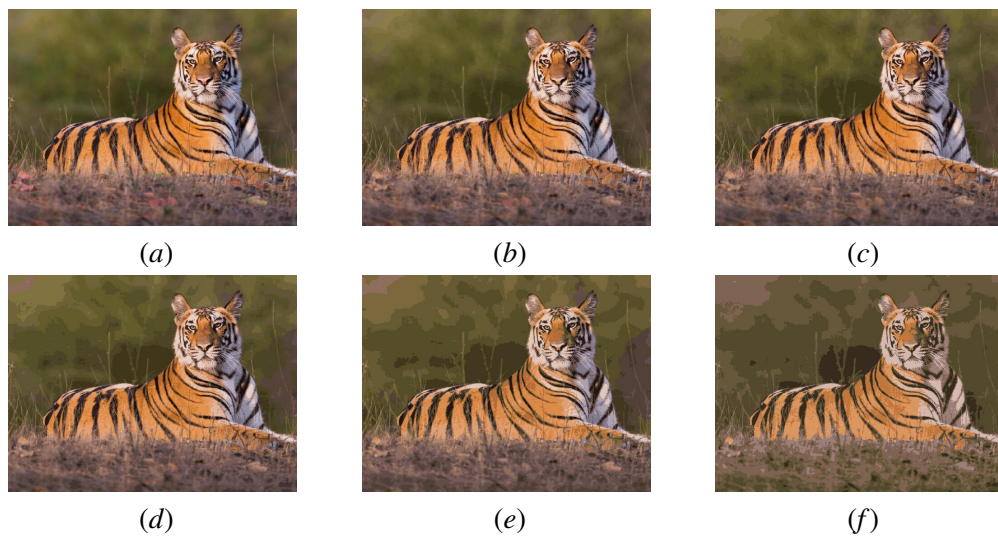


Figure 16: An example of applying the K-means algorithm for image segmentation showing **(a)** the initial image with 96,615 colors. **(b-f)** images reconstructed by $K = 128$, $K = 64$, $K = 32$, $K = 16$, and $K = 8$ colors using the K-means algorithm, respectively. This also illustrates the use of vector quantization for data compression, in which smaller values of K give higher compression at the expense of poorer image quality. Note that for this particular image, with $K = 128$, you can encode the tiger pretty well!

5.2. Vector Quantization

We can also use the result of a clustering algorithm to perform data compression¹⁸. We can apply the K-means algorithm to the problem of lossy data compression as follows. For an image of width w and height h , we have $N = w \times h$ data points. For each pixel, we store only the identity i of the cluster to which it is assigned to. We also store the values of the K cluster centers $\mu_i \in \mathbb{R}^3$. This typically requires significantly less data, since we choose $K \ll N$. Each data point is then approximated by its nearest center $\mu_i \in \mathbb{R}^3$. New data points can similarly be compressed by first finding the nearest $\mu_i \in \mathbb{R}^3$ and then storing the associated label i instead of the original data vector. This framework is often called vector quantization, and the vectors μ_i are called code-book vectors.

Remark 6 (Compression Rate) Suppose the original image has N pixels comprising (R, G, B) . Recall that each channel of an RGB image is stored with 8 bits of precision. To store the whole image, you need $24N$ bits. Now suppose we first run K-means on the image data, and then instead of storing the original pixel intensities, we store the identity of the nearest vector $\{\mu_1, \mu_2, \dots, \mu_K\}$. Because there are K such vectors, we need $\log_2(K)$ bits to encode each pixel. We must also store the code book vectors (i.e., $\mu_i \in \mathbb{R}^3$), which requires $24K$ bits if we round up to the nearest integer (8bit integers per each dimension). All in all, the total number of bits required to store the image is $24K + N \log_2(K)$. The original image shown in Figure 16 has $640 \times 480 = 307,200$ pixels and so requires $24 \times 307,200 = 7,372,800$ bits to store. By comparison, the compressed images with $K = 8$ requires $24K + N \log_2(K) = 24 \times 8 + 307,200 \times \log_2(8) = 921,792$ bits to store. This represents a [compression ratio](#) of nearly 8! We see that there is a trade-off between the degree of compression and image quality.

Being curious, you may ask yourself, isn't it better to find patterns in images that appear often and encode the image accordingly? Such patterns could be considered as small patches (e.g., 16×16) that once assembled, can create natural images reasonably well. A naive approach to do this may fail miserably because patterns in images will be slightly different, so we cannot simply count how many times a particular pattern occurs. Here is a strategy based on the K-means. We take a training set of images and cut each image into patches of fixed size and reshape them into d dimensional vectors. We then build a set of clusters out of these d -dimensional vectors. This set of clusters is our codebook (or dictionary), because we expect cluster centers to look like pieces that often occur in the images. We can now describe any new patch of an image with the cluster center closest to that piece. This means that a patch of an image is described with a number in the range $[1, 2, \dots, K]$ (where you get to choose K), and two patches that are close should be described by the same number. This strategy is known as Vector Quantization (often VQ). VQ and its variants are widely used to address a broad range of problems in computer vision, including [inpainting](#)¹⁹, image compression, and denoising, to name a few. Applications of advanced VQ techniques for image inpainting and denoising are shown in Figures 18 and 19. Focusing on inpainting for the sake of discussion, the idea here is to first learn a codebook of patches from natural images. With such a codebook at our disposal, you can reconstruct clean patches out of noisy patches (here, patches

¹⁸We distinguish between two forms of data compression, namely **lossless** and **lossy** compression. In lossless data compression, the goal is to reconstruct the original data exactly from the compressed representation. In lossy data compression, we accept some errors in the reconstruction, and in return, we can achieve higher rates of compression. Our focus in this part is on lossy compression.

¹⁹Inpainting is a conservation process where damaged, deteriorating or missing parts of an artwork are filled in to present a complete image.

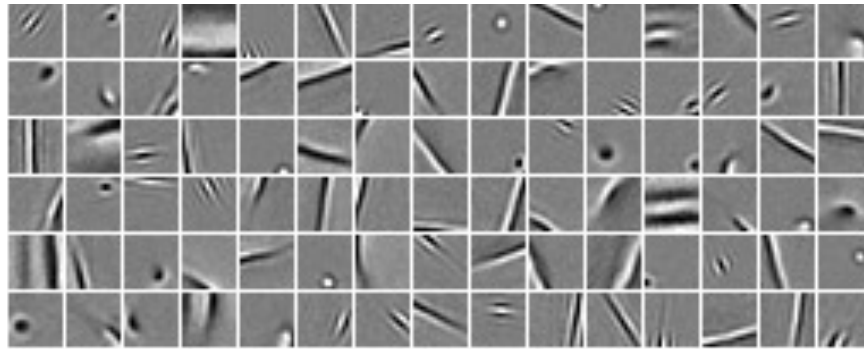


Figure 17: A dictionary obtained from image patches. Interestingly, elements of the dictionary, known as codewords, exhibit responses to edges at a different orientation, something similar to the behavior of simple and complex cells in the visual cortex. You may want to watch a video of the work of [Hubel](#) and [Wiesel](#) on understanding our visual cortex [here](#).

with red text on them). If you are interested in knowing more about the applications of VQ, you may want to go over [Mairal et al. \(2010\)](#); [Couzinie-Devry et al. \(2011\)](#).

Remark 7 *Olshausen et al. in their celebrated work reported a very interesting observation. Patches learned by VQ (not exactly VQ but a sister method) exhibit similar patterns to cells in human's visual cortex [Olshausen and Field \(1996\)](#)! See Figure 17 for an example.*

5.3. Superpixels

Images are represented as a grid of pixels. We take these $h \times w$ pixel grids and then apply algorithms to them, such as face detection and recognition. The problem is that, in many tasks, our pixel grids are by no means natural representations of images. If I just observe a single pixel from an image, I cannot say much, if any, about its semantic meaning. As of now, our technology dictates how digital images are captured and represented, simply we need to work with grids of pixels.

Definition 8 (Superpixel) *Superpixels are free-form groups of pixels, where within each group, pixels are similar in color and other low-level properties [Ren and Malik \(2003\)](#).*

Superpixels are becoming an integrated part of various techniques in computer vision such as segmentation, and object detection and tracking because of the the following desirable properties (among others)

- **Computational efficiency:** Superpixels will allow us to reduce the complexity of the images from millions of pixels to only a few hundred superpixels. Each superpixel will contain some sort of perceptual, and ideally, semantic value.
- **Perceptual meaningfulness:** Instead of only examining a single pixel in a pixel grid, which carries very little perceptual meaning, pixels that belong to a superpixel group share some sort of commonality, such as similar color or texture distribution.

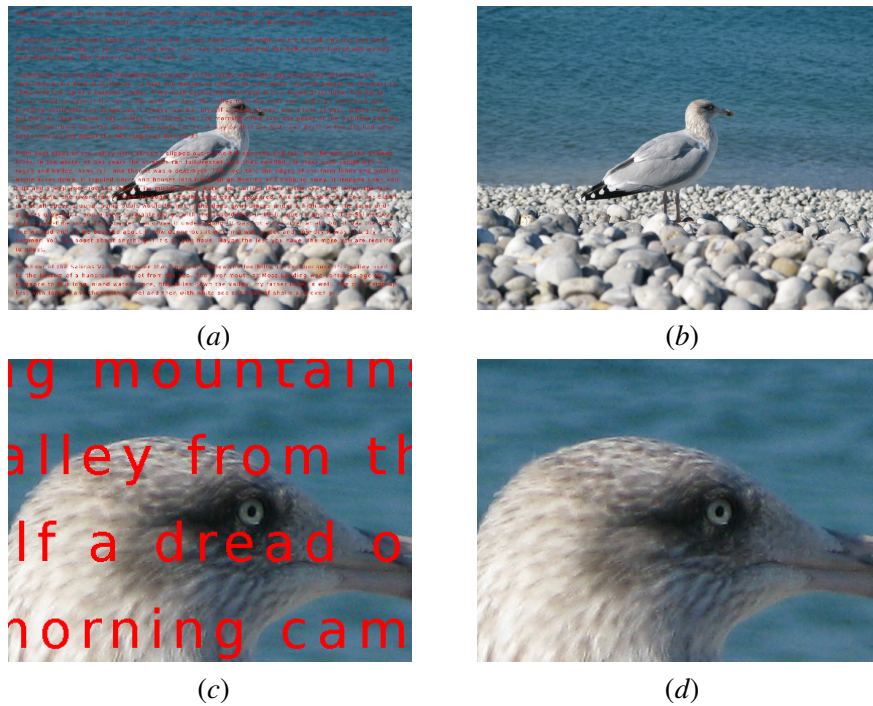


Figure 18: Inpainting example on a 12-Megapixel image. **Top:** Damaged and restored images. **Bottom:** Zooming on the damaged and restored images (Best seen in color). Figure courtesy [Mairal et al. \(2010\)](#).

Now you may ask yourself how could we obtain superpixels of an image. First, have a look again at the title page. There we showed superpixels for three images. As you may notice, this looks like some sort of clustering, albeit not with a predefined K (*i.e.*, fixed number of clusters). SLIC (Simple Linear Iterative Clustering) [Achanta et al. \(2012\)](#) is a widely adopted superpixel algorithm. The SLIC algorithm first initializes $m \times m$ cluster centers on a uniform grid on an image. Then the algorithm uses an iterative clustering algorithm to assign pixels inside patches of a predefined size (this is a hyperparameter of the SLIC algorithm) to the cluster centers, followed by updating the cluster centers. Note that images in the title page are segmented using SLIC into superpixels of size 64, 256, and 1024 pixels (approximately).

We would like to conclude this part by showing how versatile the idea of clustering is. The image below is a computer-generated image by an algorithm called VQGAN [Esser et al. \(2021\)](#). We have no intention to get into the details but in this work, the authors showed a large number of images to a Deep Neural Network (DNN). The DNN learns a codebook (think of it again as cluster centers in K-means) and then employs that to generate new images (see an example in Figure 20).

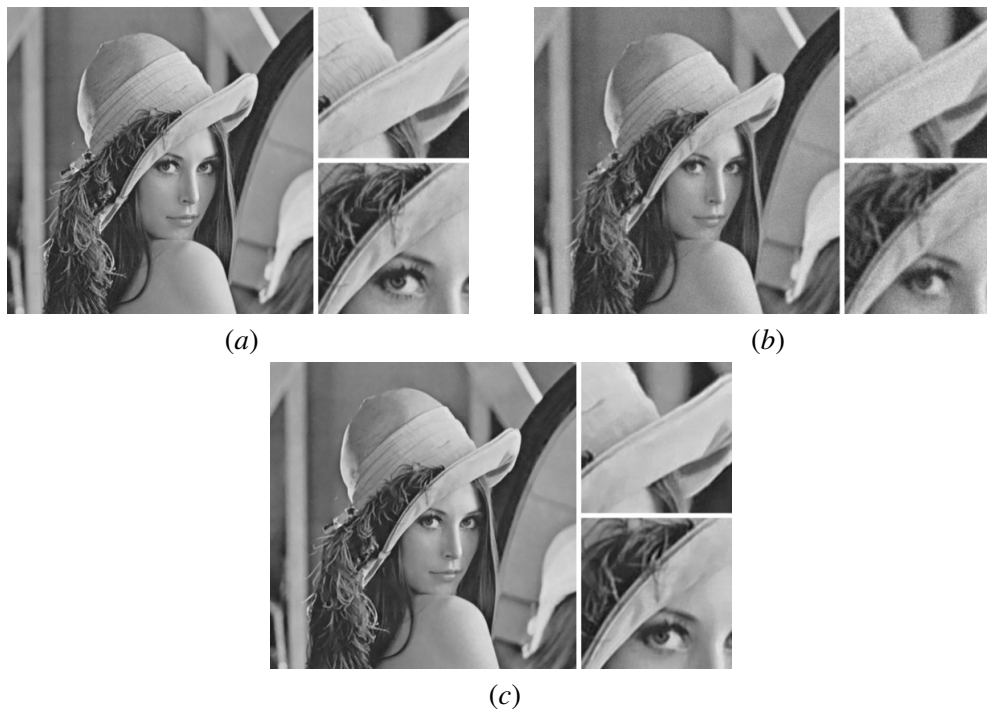


Figure 19: Examples of deblurring. **(a)** original image, **(b)** blurry image, **(c)** deblurring based on VQ. Figure courtesy [Couzinie-Devy et al. \(2011\)](#).

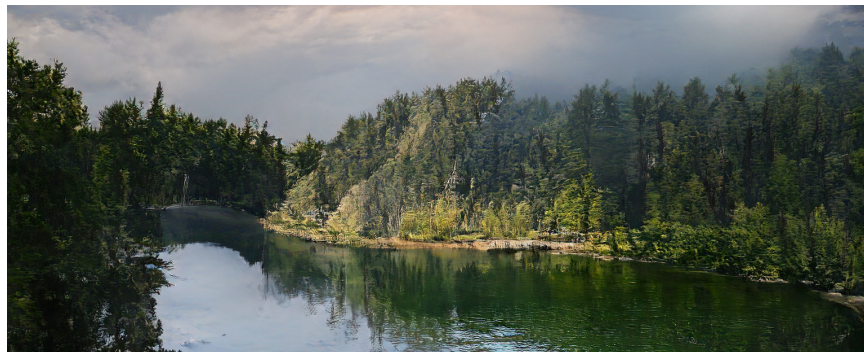


Figure 20: An image generated by VQGAN [Esser et al. \(2021\)](#).

6. Exercises

6.1. K-means

Exercise 1 *How does the choice of initial centroids in K-means clustering affect the resulting cluster assignments?*

Recall that the initial centroids can significantly impact the quality of the K-means clustering. For example, if the initial centroids are too far away from the data, it may result in several clusters becoming zombies (clusters without any points assigned to them), or it may take longer for the algorithm to converge.

Exercise 2 *What are some common methods for selecting initial centroids, and how do they compare in terms of convergence speed and cluster quality?*

Common methods for initializing K-means include **random initialization**, **initialization via data** (so you randomly pick k points out of your data for initialization), and of course, **the K-means++ algorithm**. Regarding convergence speed and cluster quality, K-means++ is generally considered the most effective method for selecting initial centroids, as it tends to lead to faster convergence and better quality clusters than random initialization.

Exercise 3 *Can K-means clustering be used to identify outliers in a dataset?*

Outliers are data points that deviate significantly from the rest of the data. One method for identifying outliers using K-means clustering is to set a threshold on the distance between each data point and its assigned cluster centroid. Data points that are farther away from their centroid than the threshold are considered to be outliers. If you are interested in outlier detection techniques, you may want to check [isolation forest](#), and metrics such as [Local Outlier Factor \(LOF\)](#).

Exercise 4 *Suppose you have a dataset in $\mathbb{R}^2$ and want to cluster the data into three clusters using K-means. After running the K-means algorithm, you obtain the final centroids $\mu_1 = (2, 3)^\top$, $\mu_2 = (4, 5)^\top$, and $\mu_3 = (8, 9)^\top$, respectively. Suppose you have a new data point at $\mathbf{x} = (3, 4)^\top$. Which cluster would this point be assigned to, based on the final cluster assignments produced by K-means?*

To determine which cluster the new data point $\mathbf{x} = (3, 4)^\top$ would be assigned to, we first need to compute the distance between the point and each cluster centroid. The distance between $\mathbf{x} = (3, 4)^\top$ and the centroid $\mu_1 = (2, 3)^\top$ is

$$d(\mathbf{x}, \mu_1) = \sqrt{(3-2)^2 + (4-3)^2} = \sqrt{2}.$$

Similarly, the distance between $\mathbf{x}$ and other centroids μ_2 and μ_3 is;

$$d(\mathbf{x}, \mu_2) = \sqrt{(3-4)^2 + (4-5)^2} = \sqrt{2}.$$

$$d(\mathbf{x}, \mu_3) = \sqrt{(3-8)^2 + (4-9)^2} = \sqrt{50}.$$

Therefore, $\mathbf{x}$ is closest to the centroid μ_1 and μ_2 , both of which have a distance of $\sqrt{2}$ from the point. Here we have a tie, and we could either randomly assign to a cluster or pick the cluster with the lowest index as the final answer.

Exercise 5 Suppose in a K -means clustering you have two centroids in $\mathbb{R}^2$ at points $\mu_1 = (x_1, y_1)$ and $\mu_2 = (x_2, y_2)$. Derive the form of the points that cannot assign to any clusters with confidence (i.e., points that are equidistant from the centroids)?

You can approach this problem in two different ways.

Approach 1. Consider $\mathbf{x} = (x, y)$. If the distance between $\mathbf{x}$ and μ_1 is equal to its distance to μ_2 , then we have

$$d(\mathbf{x}, \mu_1) = d(\mathbf{x}, \mu_2) \Rightarrow d^2(\mathbf{x}, \mu_1) = d^2(\mathbf{x}, \mu_2) .$$

Write,

$$d^2(\mathbf{x}, \mu_1) = (x - x_1)^2 + (y - y_1)^2 .$$

$$d^2(\mathbf{x}, \mu_2) = (x - x_2)^2 + (y - y_2)^2 .$$

Equating the two distances

$$\begin{aligned} (x - x_1)^2 + (y - y_1)^2 &= (x - x_2)^2 + (y - y_2)^2 \\ \Rightarrow \cancel{x^2} - 2xx_1 + x_1^2 + \cancel{y^2} - 2yy_1 + y_1^2 &= \cancel{x^2} - 2xx_2 + x_2^2 + \cancel{y^2} - 2yy_2 + y_2^2 \\ \Rightarrow 2(x_1 - x_2)x + 2(y_1 - y_2)y &= x_1^2 + y_1^2 - x_2^2 - y_2^2 . \end{aligned}$$

This is the equation of the line of equidistant from both centroids.

Approach 2. We take a geometrical approach here. The line that is equidistant from two centroids in $\mathbb{R}^2$ is the perpendicular bisector of the line segment connecting the two centroids. To find the equation of this line, we first need to find the midpoint of the line segment connecting the two centroids. The midpoint of the line segment is given by:

$$\mu = \begin{pmatrix} \frac{x_1 + x_2}{2} \\ \frac{y_1 + y_2}{2} \end{pmatrix} .$$

Next, we need to find the slope of the line segment connecting the two centroids. The slope of the line segment is given by:

$$m = \frac{y_2 - y_1}{x_2 - x_1} .$$

The slope of the perpendicular bisector of the line segment is the reciprocal slope of the line segment connecting the two centroids. Therefore, the slope of the perpendicular bisector is:

$$m = \frac{x_2 - x_1}{y_2 - y_1} .$$

Finally, we can use the point-slope form of the line equation to find the perpendicular bisector. This results in

$$y - \frac{y_1 + y_2}{2} = \frac{x_2 - x_1}{y_2 - y_1} \left(x - \frac{x_1 + x_2}{2} \right) .$$

I leave this to you to show that the two approaches give the same line.

Exercise 6 Suppose you have a dataset of n points $\mathcal{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$. Each sample $\mathbf{x}_i$ comes with a corresponding weight $w_i \in [0, 1]$, showing the importance of the sample. You want to perform K -means clustering on this dataset to group the points into k clusters. However, you also want to take into account the weights of the points in the clustering process.

Design an algorithm for performing weighted K -means clustering on this dataset. Your algorithm should take as input the dataset, the number of clusters k , and the weights of the points. It should output the k centroids that minimize the weighted sum of squared distances between the points and their assigned centroids. Explain how your algorithm differs from the standard K -means clustering algorithm and how it accounts for the weights of the points.

To perform K -means clustering, we need to address two questions:

1. **Assignment.** given the current cluster centers $\{\mu_1, \mu_2, \dots, \mu_k\}$, to which cluster a sample $\mathbf{x}$ is belong?
2. **Centroids.** if samples $\{\mathbf{x}_i\}$ are assigned to a cluster, what should be the cluster centroid?

The answer to the first question is straightforward. We compute the distance to the centroids and pick the closest cluster. Weights, in this case have no impact on our decision. As for the second question, recall the cluster centroid is the point that has the minimum total sum of squared distances in K -means. That is, the cluster center is the solution of the following optimization problem in K -means.

$$\mu^* = \arg \min_{\mu} \sum_i \|\mathbf{x}_i - \mu\|^2.$$

This problem has a very simple solution; μ is indeed the average of points. If each sample has a weight w_i , we can extend the above equation to consider the weights. In other words, to obtain the centroid, we opt to solve the following optimization problem.

$$\mu^* = \arg \min_{\mu} \sum_i w_i \|\mathbf{x}_i - \mu\|^2. \quad (25)$$

It is quite easy to show that the solution of eq. (25) is the weighted average (compute the derivative with respect to μ and set it to zero).

$$\mu^* = \frac{\sum_i w_i \mathbf{x}_i}{\sum_i w_i}. \quad (26)$$

6.2. GMM

Exercise 7 Compare and contrast the k -means algorithm and Gaussian Mixture Models (GMMs) for clustering. In what ways are they similar, and in what ways are they different?

In terms of similarities;

- Both algorithms require the user to specify the number of clusters.
- Both algorithms use an iterative approach to optimize their objective function.

As for differences;

- K-means assigns each data point to a single cluster, whereas GMMs assign each data point a probability of belonging to each cluster.
- K-means assumes that the clusters are spherical and of equal size, whereas GMMs allow for more flexible cluster shapes and sizes.
- K-means is more computationally efficient than GMMs, as it has a simpler optimization problem.
- GMMs can provide a probabilistic measure of how likely each point is to belong to each cluster, which can be useful in applications such as image segmentation.

Exercise 8 *Can you think of a clustering scenario where K-means fails but a GMM triumphs?*

Consider the following example. Suppose we have a dataset of 2D points that contains two distinct clusters, one circular and one elliptical in shape. K-means will struggle to accurately cluster these points, as it will try to fit each cluster to a circular shape. On the other hand, GMMs can accurately model these clusters by using a combination of circular and elliptical shapes.

Exercise 9 *Your friend claims that the K-means algorithm is a special case of a Gaussian mixture model. Do you agree with them? If yes, explain why and how a GMM can be turned into a K-means algorithm.*

For the sake of simplicity, I assume we are working in $\mathbb{R}$. Extension to the multivariate case is straightforward. We can derive the K-means algorithm as a limiting case of the GMM as follows.

Consider a GMM with K components in which the variance of the mixture components is given by ϵ^2 (i.e., shared variance). Also, assume $w_i = 1/K$. The likelihood of x for the j -th component is

$$\phi_k(x) = \frac{1}{\sqrt{2\pi\epsilon^2}} \exp\left(-\frac{1}{2\epsilon^2} \|x - \mu_k\|^2\right)$$

The responsibilities for a particular data point x are given by

$$\gamma_k(x) = \frac{w_k \exp\left(-\frac{1}{2\epsilon^2} \|x - \mu_k\|^2\right)}{\sum_i w_i \exp\left(-\frac{1}{2\epsilon^2} \|x - \mu_i\|^2\right)} = \frac{\exp\left(-\frac{1}{2\epsilon^2} \|x - \mu_k\|^2\right)}{\sum_i \exp\left(-\frac{1}{2\epsilon^2} \|x - \mu_i\|^2\right)}$$

Without loss of generality, assume x is closer to μ_k as compared to other components. This implies that γ_j moves towards zero faster than γ_k (why) if $\epsilon \rightarrow 0$. Since $\sum_i \gamma_i = 1$, if all $\gamma_j \rightarrow 0, j \neq k$, then $\gamma_k \rightarrow 1$. Thus, in this limit, we obtain a hard assignment of data points to clusters, just as in the K-means algorithm.

Exercise 10 Consider a 1D Gaussian Mixture Model (GMM). The Gaussian distribution (we show it by $\phi(x)$ for simplicity) is

$$\phi(x) = \frac{1}{\sqrt{2\pi}\sigma^2} \exp\left(-\frac{1}{2\sigma^2}(x-\mu)^2\right).$$

Assume we have a GMM with two components with the following parameters;

$$\begin{aligned}\mu_1 &= -1 \\ \mu_2 &= 2 \\ \sigma_1^2 &= \sigma_2^2 = 1 \\ w_1 &= \frac{2}{3} \\ w_2 &= \frac{1}{3}\end{aligned}$$

Answer the following questions.

- For $x = 0$, which component has a higher responsibility?
- We would like to obtain the decision boundary of the mixture model. The decision boundary is described by the points that have similar responsibility values with respect to both components. Obtain the decision boundary for this problem.

$$\begin{aligned}\phi_1(x=0) &= \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}\right) \\ \phi_2(x=0) &= \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{4}{2}\right)\end{aligned}$$

$$\begin{aligned}\gamma_1(x=0) &= \frac{w_1\phi_1(x=0)}{w_1\phi_1(x=0) + w_2\phi_2(x=0)} \\ &= \frac{\frac{2}{3}\frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}\right)}{\frac{2}{3}\frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}\right) + \frac{1}{3}\frac{1}{\sqrt{2\pi}} \exp\left(-\frac{4}{2}\right)} \\ &= \frac{2 \exp\left(-\frac{1}{2}\right)}{2 \exp\left(-\frac{1}{2}\right) + \exp\left(-\frac{4}{2}\right)} \\ &= \frac{1.213}{1.213 + 0.135} \approx \boxed{0.9}\end{aligned}$$

$$\gamma_2(x=0) = 1 - \gamma_1(x=0) \approx \boxed{0.1}$$

hence, the first component has a higher responsibility.

$$\begin{aligned}
\gamma_1(x) = \gamma_2(x) &\Rightarrow \frac{w_1\phi_1(x)}{w_1\phi_1(x) + w_2\phi_2(x)} = \frac{w_2\phi_2(x)}{w_1\phi_1(x) + w_2\phi_2(x)} \\
&\Rightarrow w_1\phi_1(x) = w_2\phi_2(x) \\
&\Rightarrow \frac{2}{3} \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}(x - (-1))^2\right) = \frac{1}{3} \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}(x - (2))^2\right) \\
&\Rightarrow 2 \exp\left(-\frac{1}{2}(x + 1)^2\right) = \exp\left(-\frac{1}{2}(x - 2)^2\right) \\
&\Rightarrow \log(2) - \frac{1}{2}(x + 1)^2 = -\frac{1}{2}(x - 2)^2 \\
&\Rightarrow 2 \log(2) - (x^2 + 2x + 1) = -(x^2 - 4x + 4) \\
&\Rightarrow 2 \log(2) - 2x - 1 = 4x - 4 \\
&\Rightarrow \boxed{x = \frac{2 \log(2) + 3}{6}}
\end{aligned}$$

7. You Should

The information presented in this section is intended to help you assess your understanding of the chapter content and is not necessarily representative of what you may encounter on the final exam. As always, be sure to review all course materials thoroughly in preparation for any assessments.

Remember These Terms

- Clustering
- K-Means
- Centroids
- Elbow Method
- Gaussian Mixture Model (GMMs)
- Responsibility
- E-step, M-step

Know These Facts and Procedures

- **K-means.** an iterative clustering algorithm that partitions data points into K clusters, where K is a user-defined parameter.
- The objective function of K-means is to minimize the sum of squared distances between each data point and its assigned cluster center.
- K-means is sensitive to the initialization of the cluster centers, and can converge to a suboptimal solution if the initialization is poor.

- A common technique for initializing the cluster centers is to randomly select K data points from the dataset.
- GMMs are probabilistic models that assume the data points are generated from a mixture of Gaussian distributions.
- Each component of the mixture represents a cluster, and is characterized by its mean vector and covariance matrix and an associated weight.
- GMMs can be trained using the Expectation-Maximization (EM) algorithm, which alternates between computing the responsibility of each component for each data point (E-step) and updating the parameters of each component (M-step).

Be Able to

- Compute Euclidean distance between two points in $\mathbb{R}^n$
- Compute responsibility value of components of a GMM
- Soft-assign points to their clusters in a GMM

References

- Radhakrishna Achanta, Appu Shaji, Kevin Smith, Aurelien Lucchi, Pascal Fua, and Sabine Süsstrunk. Slic superpixels compared to state-of-the-art superpixel methods. *IEEE Trans. on Pattern Analysis and Machine Intelligence*, 34(11):2274–2282, 2012.
- David Arthur and Sergei Vassilvitskii. K-means++: The advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, SODA '07, page 1027–1035, USA, 2007. Society for Industrial and Applied Mathematics. ISBN 9780898716245.
- Christopher M Bishop and Nasser M Nasrabadi. *Pattern recognition and machine learning*. Springer, 2006.
- Dorin Comaniciu and Peter Meer. Mean shift: A robust approach toward feature space analysis. *IEEE Trans. on Pattern Analysis and Machine Intelligence*, 24(5):603–619, 2002.
- Florent Couzinie-Devy, Julien Mairal, Francis Bach, and Jean Ponce. Dictionary learning for deblurring and digital zoom. *Int. Journal of Computer Vision*, 2011.
- Arthur P Dempster, Nan M Laird, and Donald B Rubin. Maximum likelihood from incomplete data via the em algorithm. *Journal of the Royal Statistical Society: Series B (Methodological)*, 39(1):1–22, 1977.
- Patrick Esser, Robin Rombach, and Bjorn Ommer. Taming transformers for high-resolution image synthesis. In *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, pages 12873–12883, 2021.
- David Forsyth and Jean Ponce. *Computer vision: A modern approach*. Prentice hall, 2011.
- Stuart Lloyd. Least squares quantization in pcm. *IEEE transactions on information theory*, 28(2):129–137, 1982.
- Julien Mairal, Francis Bach, Jean Ponce, and Guillermo Sapiro. Online learning for matrix factorization and sparse coding. *Journal of Machine Learning Research*, 11(1), 2010.
- Bruno A Olshausen and David J Field. Emergence of simple-cell receptive field properties by learning a sparse code for natural images. *Nature*, 381(6583):607–609, 1996.
- Xiaofeng Ren and Jitendra Malik. Learning a classification model for segmentation. In *Proc. Int. Conf. on Computer Vision (ICCV)*, volume 2, pages 10–10. IEEE Computer Society, 2003.
- Robert L Thorndike. Who belongs in the family. In *Psychometrika*, 1953.